CAIRO UNIVERSITY

Faculty of Computers and Artificial Intelligence
Computer Science Department

CS213: Object Oriented Programming (Programming II)
Academic Year 2026-2027 — First Semester

Assignment 2 — Part 1 Project Report

OOP Design & Multi-AI Architectural Exploration of the Vole Machine Simulator

Course Instructor: **Dr. Mohammad El-Ramly**

Student Name	Student ID	Section	Email	Responsibilities
Ahmed Mohmed Abd ElAzim	20250057	S1 | S2	20250057@stud.fci-cu.edu.eg	Initial design, Vole programs, AI prompt formulation
Ahmed Mohmed Fathi (Team Leader)	20250060	S1 | S2	20250060@stud.fci-cu.edu.eg	One AI exploration, architectural evaluation, recorded the presentation video
Ahmed Mohmed Moaud Zain	20250063	S1 | S2	20250063@stud.fci-cu.edu.eg	Three AI explorations, report synthesis & editing
Abdullah Wael Mohie	20250391	S1 | S2	20250391@stud.fci-cu.edu.eg	Final enhanced design synthesis, diagram generation & QA

Team & Submission Details	
Team Leader	Ahmed Mohmed Fathi (20250060)
Team Leader Phone	01008406180
Submission Date	26 October 2026

Table of Contents

1 Introduction & Vole Architecture Study . . . 3

1.1 Hardware Specifications . . . 3

1.2 Instruction Set Architecture . . . 3

2 Initial OOP Design . . . 3

2.1 Classes and Responsibilities . . . 4

2.2 Relationships . . . 4

2.3 UML Class Diagram . . . 4

3 Multi-AI Architectural Exploration . . . 7

3.1 The Prompt Given to Every AI . . . 7

3.2 AI #1: Google Gemini . . . 8

3.3 AI #2: OpenAI ChatGPT . . . 9

3.4 AI #3: Anthropic Claude . . . 13

3.5 AI #4: GitHub Copilot . . . 19

3.6 Comparison of the Four Designs . . . 21

4 Final Enhanced Design . . . 22

4.1 Why Claude's Design Was Selected . . . 22

4.2 What Was Enhanced from the Other AIs . . . 22

4.3 Refinement Dialog with Gemini 3.8 Flash . . . 22

4.4 Final Design Adopted by the Team . . . 24

5 Sample Vole Machine Programs . . . 28

5.1 Program 1: Sum of 1 to 5 . . . 29

5.2 Program 2: Maximum of Two Signed Numbers . . . 29

6 Video Link & Lessons Learned . . . 29

List of Figures

Figure 1. Initial design UML class diagram (11 classes) . . . 5

Figure 2. Google Gemini: package-centric architecture (AI design #1), rendered with Graphviz from the PlantUML code . . . 8

Figure 3. OpenAI ChatGPT: SRP and hardware-decoupled architecture (AI design #2) . . . 11

Figure 4. Anthropic Claude: robust, type-safe architecture (AI design #3) . . . 15

Figure 5. GitHub Copilot: minimalist 11-class architecture (AI design #4) . . . 20

Figure 6. Final design proposed by Gemini 3.8 Flash, rendered with Graphviz from the PlantUML code . . . 23

Figure 7. Final design adopted by the team: six-package modular architecture . . . 26

1 Introduction & Vole Architecture Study

The Vole machine (introduced by J. Glenn Brookshear) is an idealized von Neumann computer designed for teaching. It has the stored-program concept, general-purpose registers, arithmetic/logic operations, conditional branching and memory-mapped output. Our simulator models this machine in an object-oriented way: the hardware (memory, registers, CPU), the instruction set, the program loader and the console interface are separate classes.

1.1 Hardware Specifications

Component	Specification
Main memory	256 cells of 8 bits (addresses 00 to FF). Address **FF** is memory-mapped to the screen: writing to FF displays the value.
Registers	16 general-purpose 8-bit registers R0 to RF. R0 is also the comparison register used by the JUMP instruction.
Program counter (PC)	8-bit register holding the address of the next instruction. Each instruction occupies two cells, so PC advances by 2.
Instruction register (IR)	16-bit register holding the instruction being decoded and executed (4 hex digits).
ALU	Two's-complement addition, 8-bit floating-point addition, OR, AND, XOR and circular right rotation.
Control unit (CU)	Runs the fetch - validate - decode - execute cycle.

1.2 Instruction Set Architecture

Op	Format	Meaning
1	1RXY	LOAD R from memory[XY]
2	2RXY	LOAD R with the immediate value XY
3	3RXY	STORE R into memory[XY]. **3RFF** writes R to the screen.
4	40RS	MOVE: copy R into S
5	5RST	ADD S + T as two's complement integers, result in R
6	6RST	ADD S + T as floating-point values, result in R
7	7RST	R = S OR T
8	8RST	R = S AND T
9	9RST	R = S XOR T
A	AR0X	ROTATE R right by X bits (circular)
B	BRXY	JUMP to XY if R equals R0, otherwise continue (PC = XY)
C	C000	HALT

A program file contains one 4-digit hex instruction per line. Lines starting with `;` and empty lines are ignored, and everything after the first four hex digits of a line is ignored. The load address must be even and below FF.

2 Initial OOP Design

Before consulting any AI, the team modeled the domain by hand. The goals were a single responsibility per class, a hardware-like structure (Machine contains CPU and Memory; CPU contains registers, ALU and control unit) and a user interface that is fully separated from the machine so that a GUI can replace it later.

2.1 Classes and Responsibilities

Class	Responsibility	Key attributes	Key methods
`Register`	One 8-bit storage unit	value : uint8_t	getValue, setValue, clear
`RegisterFile`	The 16 registers R0-RF with indexed access	registers : Register[16]	getRegister, setRegister, clearAll, display
`Memory`	256 byte cells; address FF is mapped to the screen	cells : uint8_t[256]	getCell, setCell, clear, displayAsMatrix
`ALU`	Stateless arithmetic and logic	none	addTwosComplement, addFloatingPoint, bitwiseOR/AND/XOR, rotateRight
`Instruction`	Decoded 16-bit instruction	opcode, operand1-3, rawInstruction	getOpcode, getOperand1/2/3, getRaw, toString
`CU`	Fetch, decode, validate, execute	none (stateless)	fetch, decode, execute, isValidInstruction
`CPU`	Processor state; owns RegisterFile, ALU and CU	pc : uint8_t, ir : uint16_t, halted : bool	step, reset, getPC/setPC, getIR/setIR, displayState
`ProgramLoader`	Parses program files and loads them into memory	none (utility)	loadFromFile, parseInstruction, validateStartAddress, loadIntoMemory
`Machine`	Top-level machine; owns CPU and Memory	cpu, memory	loadProgram, run, stepByStep, reset, displayStatus, setMemoryCell, setRegister, getScreenOutput
`UserInterface`	All console input and output	none	displayMenu, getMenuChoice, displayMachineState, displayMemoryMatrix, waitForEnter, askRunMode, askContinue
`Simulator`	Application controller and main loop	machine, ui	start, mainLoop

2.2 Relationships

Relationship	Type	Relationship	Type
Machine - CPU, Memory	Composition	CU - ALU, RegisterFile, Memory	Dependency
CPU - RegisterFile, ALU, CU	Composition	CU - Instruction (creates)	Dependency
RegisterFile - Register (16)	Composition	Machine - ProgramLoader	Dependency
Simulator - Machine	Association	ProgramLoader - Memory	Dependency
Simulator - UserInterface	Association		

2.3 UML Class Diagram

The diagram is shown on the next page (Figure 1) and its PlantUML source follows it.

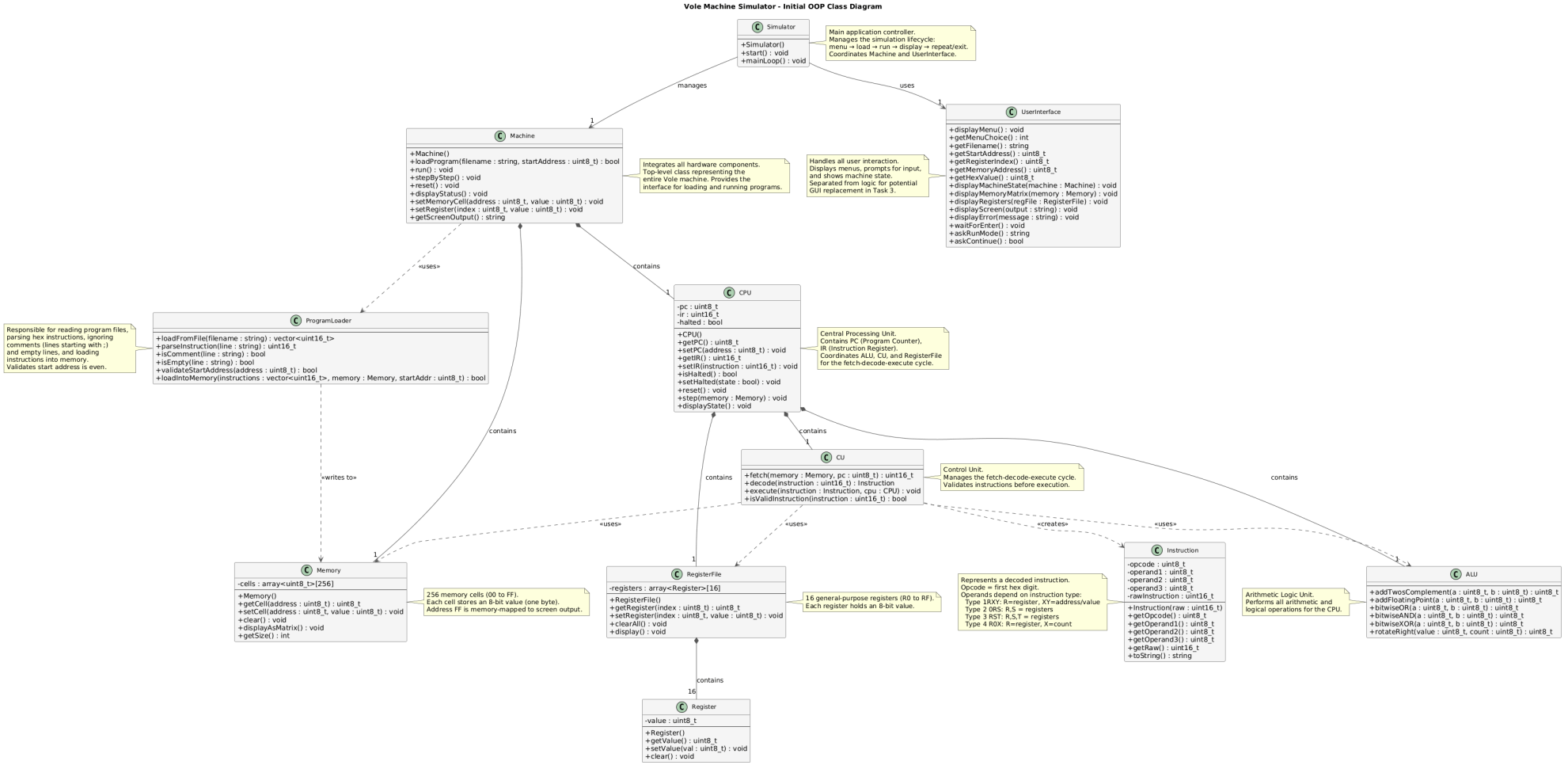


Figure 1. Initial design UML class diagram (11 classes)

PlantUML source code (initial_design.puml, 238 lines)

```
@startuml VoleMachineSimulator

skinparam classAttributeIconSize 0
skinparam classFontSize 12
skinparam classBackgroundColor #F5F5F5
skinparam classBorderColor #333333
skinparam arrowColor #555555
skinparam packageBackgroundColor #ECECEC

title Vole Machine Simulator - Initial OOP Class Diagram

' CORE DOMAIN CLASSES

class Memory {
    - cells : array<uint8_t>[256]
    --
    + Memory()
    + getCell(address : uint8_t) : uint8_t
    + setCell(address : uint8_t, value : uint8_t) : void
    + clear() : void
    + displayAsMatrix() : void
    + getSize() : int
}
note right of Memory
  256 memory cells (00 to FF).
  Each cell stores an 8-bit value (one byte).
  Address FF is memory-mapped to screen output.
end note

class Register {
    - value : uint8_t
    --
    + Register()
    + getValue() : uint8_t
    + setValue(val : uint8_t) : void
    + clear() : void
}

class RegisterFile {
    - registers : array<Register>[16]
    --
    + RegisterFile()
    + getRegister(index : uint8_t) : uint8_t
    + setRegister(index : uint8_t, value : uint8_t) : void
    + clearAll() : void
    + display() : void
}
note right of RegisterFile
  16 general-purpose registers (R0 to RF).
  Each register holds an 8-bit value.
end note

class ALU {
    --
    + addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
    + addFloatingPoint(a : uint8_t, b : uint8_t) : uint8_t
    + bitwiseOR(a : uint8_t, b : uint8_t) : uint8_t
    + bitwiseAND(a : uint8_t, b : uint8_t) : uint8_t
    + bitwiseXOR(a : uint8_t, b : uint8_t) : uint8_t
    + rotateRight(value : uint8_t, count : uint8_t) : uint8_t
}
note left of ALU
  Arithmetic Logic Unit.
  Performs all arithmetic and
  logical operations for the CPU.
end note

class CU {
    --
    + fetch(memory : Memory, pc : uint8_t) : uint16_t
    + decode(instruction : uint16_t) : Instruction
    + execute(instruction : Instruction, cpu : CPU) : void
    + isValidInstruction(instruction : uint16_t) : bool
}
note right of CU
  Control Unit.
  Manages the fetch-decode-execute cycle.
  Validates instructions before execution.
end note

class Instruction {
    - opcode : uint8_t
    - operand1 : uint8_t
    - operand2 : uint8_t
    - operand3 : uint8_t
    - rawInstruction : uint16_t
    --
    + Instruction(raw : uint16_t)
    + getOpcode() : uint8_t
    + getOperand1() : uint8_t
    + getOperand2() : uint8_t
    + getOperand3() : uint8_t
    + getRaw() : uint16_t
    + toString() : string
}
note left of Instruction
  Represents a decoded instruction.
  Opcode = first hex digit.
  Operands depend on instruction type:
    Type 1RXY: R=register, XY=address/value
    Type 2 0RS: R,S = registers
    Type 3 RST: R,S,T = registers
    Type 4 R0X: R=register, X=count
end note

class CPU {
    - pc : uint8_t
    - ir : uint16_t
    - halted : bool
    --
    + CPU()
    + getPC() : uint8_t
    + setPC(address : uint8_t) : void
    + getIR() : uint16_t
    + setIR(instruction : uint16_t) : void
    + isHalted() : bool
    + setHalted(state : bool) : void
    + reset() : void
    + step(memory : Memory) : void
    + displayState() : void
}
note right of CPU
  Central Processing Unit.
  Contains PC (Program Counter),
  IR (Instruction Register).
  Coordinates ALU, CU, and RegisterFile
  for the fetch-decode-execute cycle.
end note

class Machine {
    --
    + Machine()
    + loadProgram(filename : string, startAddress : uint8_t) :
        bool
    + run() : void
    + stepByStep() : void
    + reset() : void
    + displayStatus() : void
    + setMemoryCell(address : uint8_t, value : uint8_t) : void
    + setRegister(index : uint8_t, value : uint8_t) : void
    + getScreenOutput() : string
}
note right of Machine
  Integrates all hardware components.
  Top-level class representing the
  entire Vole machine. Provides the
  interface for loading and running programs.
end note

class ProgramLoader {
    --
    + loadFromFile(filename : string) : vector<uint16_t>
    + parseInstruction(line : string) : uint16_t
    + isComment(line : string) : bool
    + isEmpty(line : string) : bool
    + validateStartAddress(address : uint8_t) : bool
    + loadIntoMemory(instructions : vector<uint16_t>, memory :
        Memory, startAddr : uint8_t) : bool
}
note left of ProgramLoader
  Responsible for reading program files,
  parsing hex instructions, ignoring
  comments (lines starting with ;)
  and empty lines, and loading
  instructions into memory.
  Validates start address is even.
end note

class UserInterface {
    --
    + displayMenu() : void
    + getMenuChoice() : int
    + getFilename() : string
    + getStartAddress() : uint8_t
    + getRegisterIndex() : uint8_t
    + getMemoryAddress() : uint8_t
    + getHexValue() : uint8_t
    + displayMachineState(machine : Machine) : void
    + displayMemoryMatrix(memory : Memory) : void
    + displayRegisters(regFile : RegisterFile) : void
    + displayScreen(output : string) : void
    + displayError(message : string) : void
    + waitForEnter() : void
    + askRunMode() : string
    + askContinue() : bool
}
note left of UserInterface
  Handles all user interaction.
  Displays menus, prompts for input,
  and shows machine state.
  Separated from logic for potential
  GUI replacement in Task 3.
end note

class Simulator {
    --
    + Simulator()
    + start() : void
    + mainLoop() : void
}
note right of Simulator
  Main application controller.
```

```
  Manages the simulation lifecycle:
  menu → load → run → display → repeat/exit.
  Coordinates Machine and UserInterface.
end note

' RELATIONSHIPS

' Composition: Machine owns its components
Machine *-- "1" CPU : contains
Machine *-- "1" Memory : contains

' Composition: CPU owns its sub-components
CPU *-- "1" RegisterFile : contains
CPU *-- "1" ALU : contains
CPU *-- "1" CU : contains

' Composition: RegisterFile owns Registers
RegisterFile *-- "16" Register : contains

' Association: CU creates Instructions
CU ..> Instruction : <<creates>>

' Association: CU uses ALU for execute
CU ..> ALU : <<uses>>
CU ..> RegisterFile : <<uses>>
CU ..> Memory : <<uses>>

' Association: Machine uses ProgramLoader
Machine ..> ProgramLoader : <<uses>>

' Association: Simulator coordinates Machine and UI
Simulator --> "1" Machine : manages
Simulator --> "1" UserInterface : uses

' ProgramLoader writes to Memory
ProgramLoader ..> Memory : <<writes to>>

@enduml
```

3 Multi-AI Architectural Exploration

Four AI models were consulted separately, each in its own chat session and with the identical prompt (Section 3.1). The prompt contains the machine description, the instruction set, the simulator requirements and our initial design suggestion, but not the assignment document. For every model we report its response (its design choices and reasoning), the UML diagram rendered from its PlantUML code, and the code itself.

3.1 The Prompt Given to Every AI

````
# Vole Machine Simulator — Design Task for AI

## Machine Description

The Vole machine is a simple computer with the following
    architecture:

- **Memory**: 256 cells (addresses 00 to FF hex), each storing
    one byte (8 bits). Address FF is memory-mapped to screen
    output — writing to FF displays the value on screen.
- **Registers**: 16 general-purpose registers (R0 to RF), each
    holding 8 bits.
- **Program Counter (PC)**: An 8-bit register pointing to the
    address of the next instruction to fetch.
- **Instruction Register (IR)**: A 16-bit register holding the
    instruction currently being executed.
- **Instructions**: Each instruction is 16 bits (4 hex digits,
    occupying 2 consecutive memory cells). The first hex digit
    is the opcode.

## Instruction Set

| Opcode | Format | Description |
|--------|--------|-------------|
| 1 | RXY | LOAD register R with the value from memory[XY] |
| 2 | RXY | LOAD register R with the immediate value XY |
| 3 | RXY | STORE register R into memory[XY]. Special case: 3RFF
    writes to screen |
| 4 | 0RS | MOVE (copy) the value in register R to register S |
| 5 | RST | ADD registers S and T as two's complement integers,
    store result in R |
| 6 | RST | ADD registers S and T as floating-point values,
    store result in R |
| 7 | RST | Bitwise OR registers S and T, store result in R |
| 8 | RST | Bitwise AND registers S and T, store result in R |
| 9 | RST | Bitwise XOR registers S and T, store result in R |
| A | R0X | ROTATE register R right by X bits (circular
    rotation) |
| B | RXY | JUMP to address XY if register R equals register 0
    (R0). Otherwise continue normally |
| C | 000 | HALT execution |

## Simulator Requirements

Design an OOP simulator for this machine that:

1. Offers a menu of choices to the user
2. Allows loading a program from a file into memory at a
    specific start address (must be even: 00, 02, 04, ..., FE)
3. Allows the user to manually overwrite a register value or a
    memory cell value
4. Sets the Program Counter (PC) to the start memory location of
    the loaded program
5. Displays memory contents as a 16×16 matrix, and displays IR
    and all 16 registers
6. Fetches instructions step by step into the IR, validates
    them, decodes them, executes them, and updates PC
7. Lets the user choose to run the program all at once or step
    by step
8. If running all at once: displays memory, PC, IR, registers,
    and screen output after the program finishes
9. If step by step: displays all the above after each
    instruction, then waits for the user to press Enter
10. After execution, asks the user if they want to exit or load
    another program

## Program File Format

Program files contain hex instructions as strings, one per line.
    Comments start with `;` and are ignored. Empty lines are
    ignored. Only the first 4 hex characters of each line are
    the instruction; the rest of the line is ignored.

Example:
```
; Program: Add 5 + 7 = 12, display result
2005 ; Load 05 into R0
2107 ; Load 07 into R1
5201 ; Add R0 + R1, result in R2
32FF ; Store R2 to screen
C000 ; Halt
```

## My Initial Design Suggestion

I suggest the following classes for the simulator:

1. **Register** — represents a single 8-bit register (value,
    get/set)
2. **RegisterFile** — manages the 16 registers, provides indexed
    access
3. **Memory** — 256-byte array, read/write by address, display
    as matrix
4. **ALU** — performs arithmetic (add int, add float) and logic
    (OR, AND, XOR, rotate) operations
5. **Instruction** — decoded instruction with opcode and
    operands parsed from a 16-bit value
6. **CU (Control Unit)** — implements fetch, decode, validate,
    and execute steps
7. **CPU** — contains PC, IR, halt flag; owns RegisterFile, ALU,
    CU; runs the fetch-decode-execute cycle
8. **ProgramLoader** — reads program files, parses instructions,
    loads them into memory
9. **Machine** — top-level hardware class; owns CPU and Memory
10. **UserInterface** — handles all console I/O (menus, prompts,
    display)
11. **Simulator** — main application controller; coordinates
    Machine and UserInterface

Relationships:
- Machine OWNS (composition) CPU and Memory
- CPU OWNS (composition) RegisterFile, ALU, CU
- RegisterFile OWNS (composition) 16 Registers
- CU USES (dependency) ALU, RegisterFile, Memory
- Machine USES ProgramLoader
- Simulator HAS Machine and UserInterface

## Task

Based on the machine description, simulator requirements, and my
    initial design suggestion above:

1. Produce a complete OOP class diagram design in PlantUML code
2. Include all classes with their attributes (with types) and
    methods (with parameters and return types)
3. Show all relationships (composition, association, dependency)
    between classes
4. Feel free to improve, restructure, or add classes if you
    think it makes the design better
5. Explain your design choices
````

3.2 AI #1: Google Gemini

ORIGINAL RESPONSE

Hello! I have carefully analyzed your Vole Machine Simulator requirements and reviewed your 11-class initial design.

Critical Critique of Your Initial Design

Your initial decomposition is functionally sound, but from a software architecture and C++ performance standpoint, it suffers from two major issues:

1. **Over-fragmentation (Primitive Obsession):** Creating an independent `Register` class just to hold a single `uint8_t` introduces needless object overhead, memory indirection, and pointer hops. A unified `RegisterFile` managing an internal array `uint8_t registers[16]` is vastly cleaner, faster, and idiomatic in C++.
2. **Coupling between CU and CPU:** In micro-architectural simulation, separating `CU` and `CPU` into two peer communicating classes creates circular dependencies and tight back-and-forth function calls. Merging the execution loop into the `CPU` makes the processor an autonomous engine while keeping the design cohesive.
3. **Lack of Modular Packaging:** Without UML package boundaries, an 11-class diagram looks cluttered. Grouping classes into functional packages (`Hardware`, `InstructionSet`, `Processor`, `Loader`, `Application`) provides immediate architectural clarity.

Here is my recommended **PlantUML Class Diagram** embodying this architecture (the PlantUML code of the response is shown in the "PlantUML source code" part of this section).

UML CLASS DIAGRAM

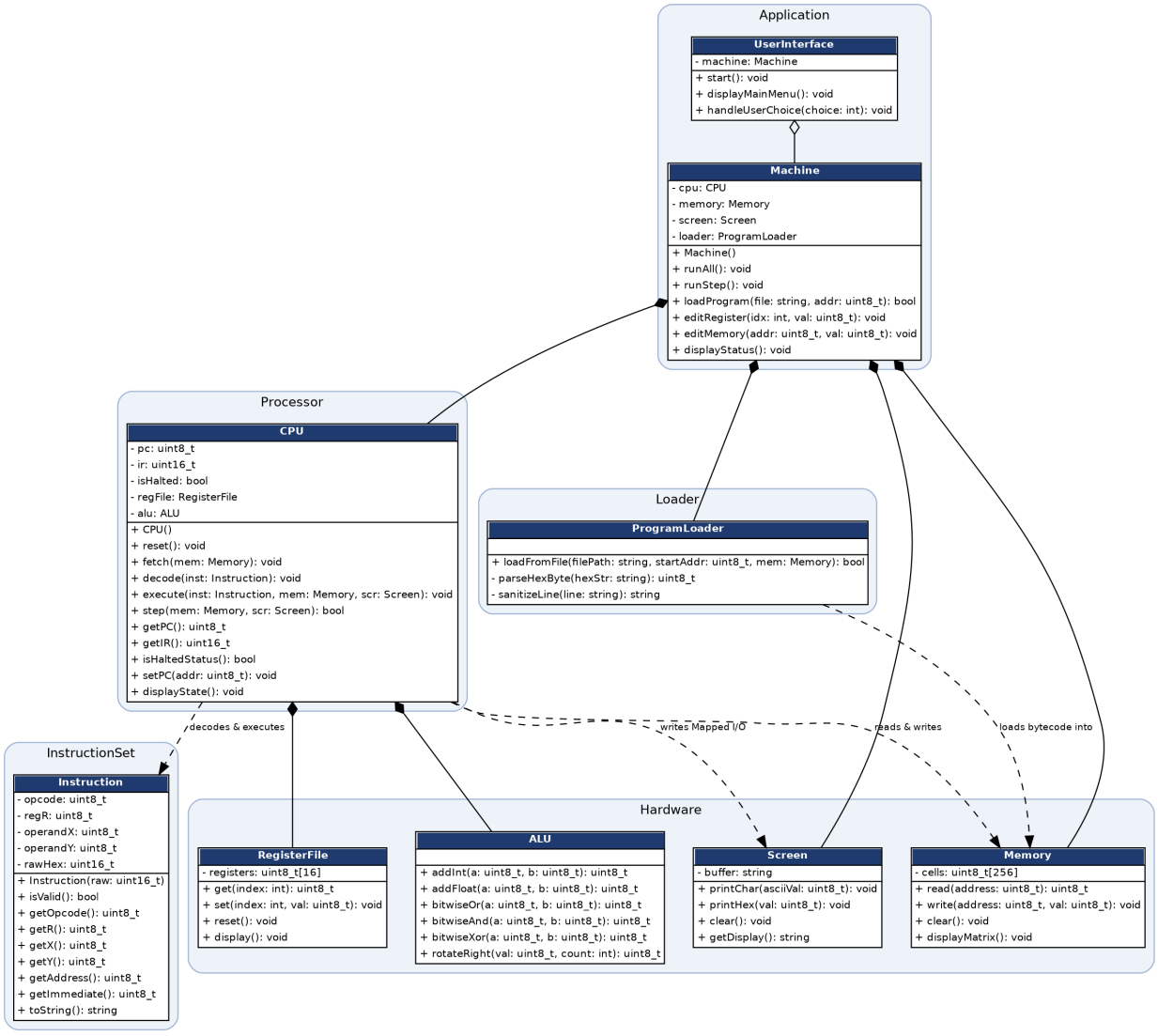


Figure 2. Google Gemini: package-centric architecture (AI design #1), rendered with Graphviz from the PlantUML code

PlantUML source code (Gemini's PlantUML code, 125 lines)

```
@startuml
title Vole Machine Simulator - Gemini Architecture Design (AI
    #1)

skinparam classAttributeIconSize 0
skinparam packageStyle rectangle

package "Hardware" {
    class RegisterFile {
        - registers: uint8_t[16]
        + get(index: int): uint8_t
        + set(index: int, val: uint8_t): void
        + reset(): void
        + display(): void
    }

    class Memory {
        - cells: uint8_t[256]
        + read(address: uint8_t): uint8_t
        + write(address: uint8_t, val: uint8_t): void
        + clear(): void
        + displayMatrix(): void
    }

    class ALU {
        + addInt(a: uint8_t, b: uint8_t): uint8_t
        + addFloat(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseOr(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseAnd(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseXor(a: uint8_t, b: uint8_t): uint8_t
        + rotateRight(val: uint8_t, count: int): uint8_t
    }

    class Screen {
        - buffer: string
        + printChar(asciiVal: uint8_t): void
        + printHex(val: uint8_t): void
        + clear(): void
        + getDisplay(): string
    }
}

package "InstructionSet" {
    class Instruction {
        - opcode: uint8_t
        - regR: uint8_t
        - operandX: uint8_t
        - operandY: uint8_t
        - rawHex: uint16_t
        + Instruction(raw: uint16_t)
        + isValid(): bool
        + getOpcode(): uint8_t
        + getR(): uint8_t
        + getX(): uint8_t
        + getY(): uint8_t
        + getAddress(): uint8_t
        + getImmediate(): uint8_t
        + toString(): string
    }
}

package "Processor" {
    class CPU {
        - pc: uint8_t
        - ir: uint16_t
        - isHalted: bool
        - regFile: RegisterFile
        - alu: ALU
        + CPU()
        + reset(): void
        + fetch(mem: Memory): void
        + decode(inst: Instruction): void
        + execute(inst: Instruction, mem: Memory, scr: Screen):
            void
        + step(mem: Memory, scr: Screen): bool
        + getPC(): uint8_t
        + getIR(): uint16_t
        + isHaltedStatus(): bool
        + setPC(addr: uint8_t): void
        + displayState(): void
    }
}

package "Loader" {
    class ProgramLoader {
        + loadFromFile(filePath: string, startAddr: uint8_t,
            mem: Memory): bool
        - parseHexByte(hexStr: string): uint8_t
        - sanitizeLine(line: string): string
    }
}

package "Application" {
    class Machine {
        - cpu: CPU
        - memory: Memory
        - screen: Screen
        - loader: ProgramLoader
        + Machine()
        + runAll(): void
        + runStep(): void
        + loadProgram(file: string, addr: uint8_t): bool
        + editRegister(idx: int, val: uint8_t): void
        + editMemory(addr: uint8_t, val: uint8_t): void
        + displayStatus(): void
    }

    class UserInterface {
        - machine: Machine
        + start(): void
        + displayMainMenu(): void
        + handleUserChoice(choice: int): void
    }
}

' Relationships
CPU *-- RegisterFile
CPU *-- ALU
Machine *-- CPU
Machine *-- Memory
Machine *-- Screen
Machine *-- ProgramLoader
UserInterface o-- Machine
CPU ..> Instruction : decodes & executes
CPU ..> Memory : reads & writes
CPU ..> Screen : writes Mapped I/O
ProgramLoader ..> Memory : loads bytecode into
@enduml
```

3.3 AI #2: OpenAI ChatGPT

ORIGINAL RESPONSE (ENGLISH TRANSLATION OF THE ARABIC REPLY)

Opening remarks

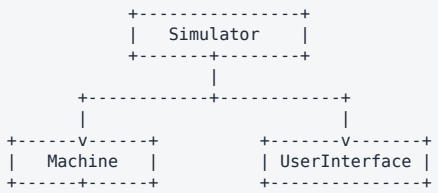
Sure, Abu Zain. Your initial design is **very good in terms of dividing responsibilities**, but there are a few points I prefer to modify:

- Make `CU` responsible for the execution cycle, while `CPU` is the interface that manages the processor state.
- Add `Screen` and `Program`/`ProgramLoader` more clearly, because `FF` is not an ordinary memory cell; it is **memory-mapped I/O**.
- Make `Instruction` responsible for **parsing/decoding**, while `CU` is responsible for executing it.
- Adding `ExecutionResult` is not necessary, so I will keep the design close to what is required instead of complicating it.

1. PlantUML class diagram

The response contains the complete PlantUML code of the design (shown in the "PlantUML source code" part of this section).

2. Basic shape of the relationships



```
                  |
       +----------+--------------+
       |          |              |
 +-----v-----+ +--v--------+ +---v---------+
 |    CPU    | |  Memory   | |   Screen    |
 +-----+-----+ +-----------+ +-------------+
       |
 +-----+--------------+
 |     |              |
 v     v              v
RF    ALU            CU  --uses--> ALU, RF, Memory
 |
 v
16 Registers
```

3. Why separate CPU from CU?

This is the most important point in the design. The **CPU** represents the processor state: `PC`, `IR`, `halted`, the registers, the ALU and the control unit. The **CU** is responsible for interpreting and executing instructions (fetch, decode, validate, execute).

For example, for the instruction `5201`: the CPU says "fetch instruction"; the CU takes `5201`, analyzes it into `Opcode = 5, R = 2, S = 0, T = 1`, then executes `R2 = R0 + R1` using the `ALU`. This keeps the responsibilities very clear.

4. Why is the Instruction class important?

Instead of having dozens of hex-digit operations inside `CPU`, `Instruction` converts `5201` into `opcode = ADD_INTEGER, R = 2, S = 0, T = 1` (5 is ADD, 2 is the destination R2, 0 and 1 are the source registers). For `A23F` it becomes `opcode = ROTATE, R = 2, X = F`. This way the `CU` does not care how the digits are extracted from the instruction.

5. An important point: Screen

I added `Screen` because `3RFF` is not just `memory[FF] = register[R]`; it has a side effect: the register value goes to memory address FF and then to the screen output. For example, `32FF` with `R2 = 0C` produces `Screen: 0C`. A separate `Screen` object makes the design cleaner.

6. ProgramLoader vs Program

I separated them on purpose. `Program` represents the program itself (its list of instructions: `2005, 2107, 5201, 32FF, C000`). `ProgramLoader` is responsible for opening the file, ignoring comments and empty lines, taking the first 4 hex characters, validation, converting lines to instructions, and loading them into Memory: `ProgramLoader -> Program -> Memory`.

7. Why Machine?

`Machine` represents the Vole computer itself. The user or the `Simulator` does not need to know the details of CPU, Memory, ALU, CU, RegisterFile and Screen; it just calls `machine.loadProgram(...)`, `machine.run()`, `machine.step()`, `machine.setRegister(...)`, `machine.setMemory(...)`. So `Machine` is a facade/aggregate of the hardware.

8. The role of Simulator

There is an important separation here. **Machine** cares about how the computer works. **UserInterface** cares about how to talk to the user. **Simulator** cares about what should happen and when. Therefore no `cin` or `cout` ends up inside `CPU`, `Memory` or `ALU`, which is very important in OOP design.

9. Instruction execution cycle

The design allows a clear cycle: `PC -> CU.fetch() -> Memory -> Instruction -> decode() -> validate() -> execute() -> update PC`. For example, with `PC = 00`, `Memory[00] = 52`, `Memory[01] = 01`: `IR = 5201`; decode gives `Opcode = 5, R = 2, S = 0, T = 1`; `ALU.addInteger(R0, R1)`; `R2 = result`; `PC = PC + 2`. For an instruction such as `B105`, the CU checks whether `R1 == R0`: if yes then `PC = 05`, otherwise `PC = PC + 2`.

10. Important note about the PC

I recommend that `pc` be a `byte` and not an `int`, because the PC in the Vole machine is really 8-bit. The `IR` should be `uint16_t` in C++, and the Register and Memory cell should be `uint8_t`. So if the project is done in C++, use `#include <cstdint>` with `uint8_t` and `uint16_t` in the final design instead of `byte` and `int`.

11. A small improvement to the original design

Your design was `Machine -> CPU, Memory`. I widened it to `Machine -> CPU, Memory, Screen`, with `CPU -> RegisterFile, ALU, CU`, and the CU uses `ALU, RegisterFile, Memory, Screen`. In my opinion this is the **best level of abstraction for the project**: not more complicated than required, and at the same time it shows real understanding of OOP and computer architecture, not just a set of classes that split the code. **If the project is required as an OOP assignment, this design is very suitable as a baseline for the actual C++ implementation.**

UML CLASS DIAGRAM

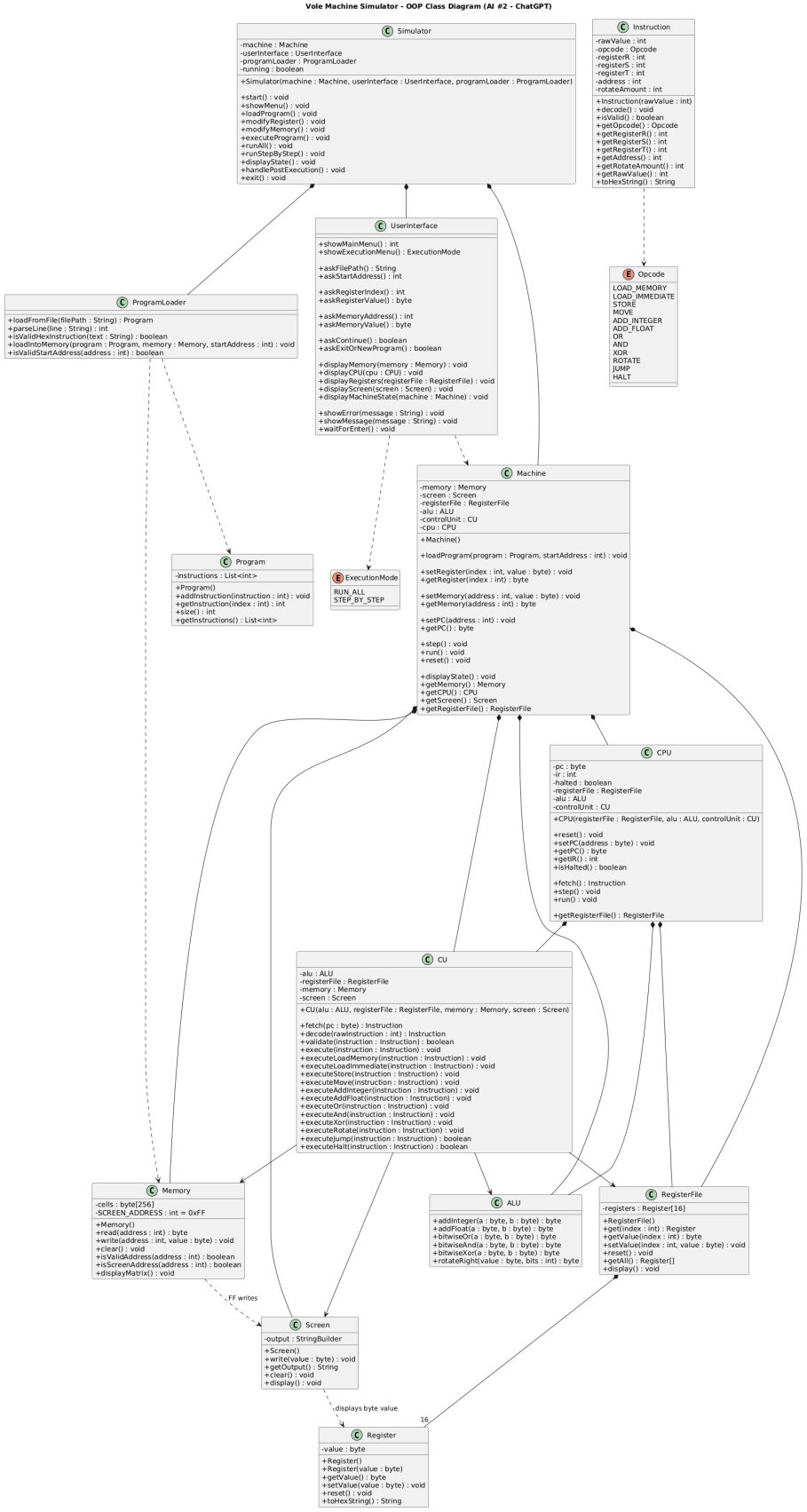


Figure 3. OpenAI ChatGPT: SRP and hardware-decoupled architecture (AI design #2)

PlantUML source code (ai2_chatgpt_design.puml, 312 lines)

```
@startuml
title Vole Machine Simulator - OOP Class Diagram (AI #2 -
    ChatGPT)

skinparam classAttributeIconSize 0
skinparam packageStyle rectangle

' ENUMS

enum ExecutionMode {
    RUN_ALL
    STEP_BY_STEP
}

enum Opcode {
    LOAD_MEMORY
    LOAD_IMMEDIATE
    STORE
    MOVE
    ADD_INTEGER
    ADD_FLOAT
    OR
    AND
    XOR
    ROTATE
    JUMP
    HALT
}

' REGISTER

class Register {
    - value : byte

    + Register()
    + Register(value : byte)
    + getValue() : byte
    + setValue(value : byte) : void
    + reset() : void
    + toHexString() : String
}

' REGISTER FILE

class RegisterFile {
    - registers : Register[16]

    + RegisterFile()
    + get(index : int) : Register
    + getValue(index : int) : byte
    + setValue(index : int, value : byte) : void
    + reset() : void
    + getAll() : Register[]
    + display() : void
}

RegisterFile *-- "16" Register

' MEMORY

class Memory {
    - cells : byte[256]
    - SCREEN_ADDRESS : int = 0xFF

    + Memory()
    + read(address : int) : byte
    + write(address : int, value : byte) : void
    + clear() : void
    + isValidAddress(address : int) : boolean
    + isScreenAddress(address : int) : boolean
    + displayMatrix() : void
}

' SCREEN

class Screen {
    - output : StringBuilder

    + Screen()
    + write(value : byte) : void
    + getOutput() : String
    + clear() : void
    + display() : void
}

Memory ..> Screen : FF writes
Screen ..> Register : displays byte value

' INSTRUCTION

class Instruction {
    - rawValue : int
    - opcode : Opcode
    - registerR : int
    - registerS : int
    - registerT : int
    - address : int
    - rotateAmount : int

    + Instruction(rawValue : int)
    + decode() : void
    + isValid() : boolean
    + getOpcode() : Opcode
    + getRegisterR() : int
    + getRegisterS() : int
    + getRegisterT() : int
    + getAddress() : int
    + getRotateAmount() : int
    + getRawValue() : int
    + toHexString() : String
}

Instruction ..> Opcode

' ALU

class ALU {
    + addInteger(a : byte, b : byte) : byte
    + addFloat(a : byte, b : byte) : byte
    + bitwiseOr(a : byte, b : byte) : byte
    + bitwiseAnd(a : byte, b : byte) : byte
    + bitwiseXor(a : byte, b : byte) : byte
    + rotateRight(value : byte, bits : int) : byte
}

' CONTROL UNIT

class CU {
    - alu : ALU
    - registerFile : RegisterFile
    - memory : Memory
    - screen : Screen

    + CU(alu : ALU, registerFile : RegisterFile, memory :
       Memory, screen : Screen)

    + fetch(pc : byte) : Instruction
    + decode(rawInstruction : int) : Instruction
    + validate(instruction : Instruction) : boolean
    + execute(instruction : Instruction) : void
    + executeLoadMemory(instruction : Instruction) : void
    + executeLoadImmediate(instruction : Instruction) : void
    + executeStore(instruction : Instruction) : void
    + executeMove(instruction : Instruction) : void
    + executeAddInteger(instruction : Instruction) : void
    + executeAddFloat(instruction : Instruction) : void
    + executeOr(instruction : Instruction) : void
    + executeAnd(instruction : Instruction) : void
    + executeXor(instruction : Instruction) : void
    + executeRotate(instruction : Instruction) : void
    + executeJump(instruction : Instruction) : boolean
    + executeHalt(instruction : Instruction) : boolean
}

CU --> ALU
CU --> RegisterFile
CU --> Memory
CU --> Screen

' CPU

class CPU {
    - pc : byte
    - ir : int
    - halted : boolean
    - registerFile : RegisterFile
    - alu : ALU
    - controlUnit : CU

    + CPU(registerFile : RegisterFile, alu : ALU, controlUnit :
       CU)

    + reset() : void
    + setPC(address : byte) : void
    + getPC() : byte
    + getIR() : int
    + isHalted() : boolean

    + fetch() : Instruction
    + step() : void
    + run() : void

    + getRegisterFile() : RegisterFile
}

CPU *-- RegisterFile
CPU *-- ALU
CPU *-- CU

' PROGRAM

class Program {
    - instructions : List<int>

    + Program()
    + addInstruction(instruction : int) : void
    + getInstruction(index : int) : int
    + size() : int
    + getInstructions() : List<int>
}

' PROGRAM LOADER
```

```
class ProgramLoader {
    + loadFromFile(filePath : String) : Program
    + parseLine(line : String) : int
    + isValidHexInstruction(text : String) : boolean
    + loadIntoMemory(program : Program, memory : Memory,
        startAddress : int) : void
    + isValidStartAddress(address : int) : boolean
}

ProgramLoader ..> Program
ProgramLoader ..> Memory

' MACHINE

class Machine {
    - memory : Memory
    - screen : Screen
    - registerFile : RegisterFile
    - alu : ALU
    - controlUnit : CU
    - cpu : CPU

    + Machine()

    + loadProgram(program : Program, startAddress : int) : void

    + setRegister(index : int, value : byte) : void
    + getRegister(index : int) : byte

    + setMemory(address : int, value : byte) : void
    + getMemory(address : int) : byte

    + setPC(address : int) : void
    + getPC() : byte

    + step() : void
    + run() : void
    + reset() : void

    + displayState() : void
    + getMemory() : Memory
    + getCPU() : CPU
    + getScreen() : Screen
    + getRegisterFile() : RegisterFile
}

Machine *-- Memory
Machine *-- Screen
Machine *-- RegisterFile
Machine *-- ALU
Machine *-- CU
Machine *-- CPU

' USER INTERFACE

class UserInterface {
    + showMainMenu() : int
    + showExecutionMenu() : ExecutionMode

    + askFilePath() : String
    + askStartAddress() : int

    + askRegisterIndex() : int
    + askRegisterValue() : byte

    + askMemoryAddress() : int
    + askMemoryValue() : byte

    + askContinue() : boolean
    + askExitOrNewProgram() : boolean

    + displayMemory(memory : Memory) : void
    + displayCPU(cpu : CPU) : void
    + displayRegisters(registerFile : RegisterFile) : void
    + displayScreen(screen : Screen) : void
    + displayMachineState(machine : Machine) : void

    + showError(message : String) : void
    + showMessage(message : String) : void
    + waitForEnter() : void
}

UserInterface ..> Machine
UserInterface ..> ExecutionMode

' SIMULATOR

class Simulator {
    - machine : Machine
    - userInterface : UserInterface
    - programLoader : ProgramLoader
    - running : boolean

    + Simulator(machine : Machine, userInterface :
        UserInterface, programLoader : ProgramLoader)

    + start() : void
    + showMenu() : void
    + loadProgram() : void
    + modifyRegister() : void
    + modifyMemory() : void
    + executeProgram() : void
    + runAll() : void
    + runStepByStep() : void
    + displayState() : void
    + handlePostExecution() : void
    + exit() : void
}

Simulator *-- Machine
Simulator *-- UserInterface
Simulator *-- ProgramLoader

@enduml
```

3.4 AI #3: Anthropic Claude

SUMMARY OF THE RESPONSE AND DESIGN CHOICES

Claude's Design Philosophy

Claude produced the **most comprehensive and production-ready** design of the three AIs, focusing on:

- **Exception handling** with a dedicated hierarchy
- **Immutable value objects** (Instruction)
- **Parameter objects** (ExecutionContext) to reduce coupling
- **Utility classes** (MiniFloat) for complex operations
- **Validation separation** (InstructionValidator)
- **Clear layering**: Hardware -> Machine -> Application

Key Design Decisions by Claude

1. Exception Hierarchy (UNIQUE)

Claude introduced a `VoleException` base class with 4 specialized exceptions:

```
VoleException
├── InvalidInstructionException  (bad opcode or operands)
├── InvalidAddressException      (out-of-range memory access)
├── ProgramFormatException       (malformed file)
└── ProgramTooLargeException     (program overflows memory)
```

Rationale: Different failure modes need different handling. `Simulator` can catch `VoleException` at the top level and show friendly error messages via `UserInterface.showError()`. This is more robust than returning bool or error codes.

2. `MiniFloat` Utility Class (UNIQUE)

Opcode 6 adds floating-point numbers in 8-bit format (1 sign + 3 exponent + 4 mantissa, excess-4). Claude isolated the encode/decode logic in a static `MiniFloat` utility class. **Rationale**: Keeps ALU clean. The floating-point format is complex enough to

deserve its own unit-testable class.

3. `InstructionValidator` as Separate Class

Like Copilot's design, but more detailed:

- `validateMove()` — ensures second nibble is 0 in `40RS`
- `validateRotate()` — ensures middle nibble is 0 in `AR0X`
- `validateHalt()` — ensures operands are `000` in `C000`

Rationale: Validation rules are complex and instruction-specific. Separating them from CU follows SRP.

4. `Instruction` as Immutable Value Object

Claude marked Instruction as `<<value object>>`:

- Parsed once in constructor, never modified
- Accessors like `getR()`, `getS()`, `getT()`, `getXY()` provide semantic access
- `getX()` extracts just the X nibble from XY (for ROTATE)

Rationale: Immutability prevents bugs. Value object pattern makes Instruction safe to pass around.

5. `ExecutionContext` as Parameter Object

Same concept as Copilot's design:

```
ExecutionContext {
    registers: RegisterFile&
    memory: Memory&
    alu: ALU&
    cpu: CPU&    ← added CPU access for PC/halt
}
```

Rationale: CU needs access to many components during execute. One context object avoids 5+ parameter methods and keeps CU stateless.

6. `Opcode` Enum with Methods

Claude's Opcode enum has:

- `fromNibble(n: int): Opcode` — converts raw nibble to enum
- `isValid(n: int): boolean` — checks if nibble is a valid opcode

Rationale: Encapsulates opcode knowledge in one place.

7. `MenuOption` Enum (UNIQUE)

Separate enum for menu choices: LOAD_PROGRAM, EDIT_REGISTER, EDIT_MEMORY, DISPLAY_STATE, RUN_ALL, RUN_STEP, EXIT. **Rationale**: Decouples UI input parsing from Simulator logic. No raw string/int comparisons.

8. `Memory` Takes `Screen*` in Constructor

Memory holds a pointer to Screen. When `write(0xFF, val)` is called, Memory forwards to `Screen.print(val)`. **Rationale**: Memory-mapped I/O is handled transparently. CU doesn't need to know about screen mapping.

9. `snapshot()` Methods

Both `RegisterFile.snapshot()` and `Memory.snapshot()` return copies of their entire state. **Rationale**: Safe state display — UI gets a copy, not a reference to live data.

10. Execution Flow (Subtle Bug Fix)

Claude explicitly noted that PC is advanced **before** execute:

```
1. CU.fetch(pc, memory) → IR
2. InstructionValidator.validate(IR)
3. CU.decode(IR) → Instruction
4. PC += 2                          ← BEFORE execute
5. CU.execute(inst, ctx)            ← JUMP can overwrite PC
```

Rationale: If PC is incremented AFTER execute, a successful JUMP would be overwritten by PC += 2. This is a **real bug** that Claude caught.

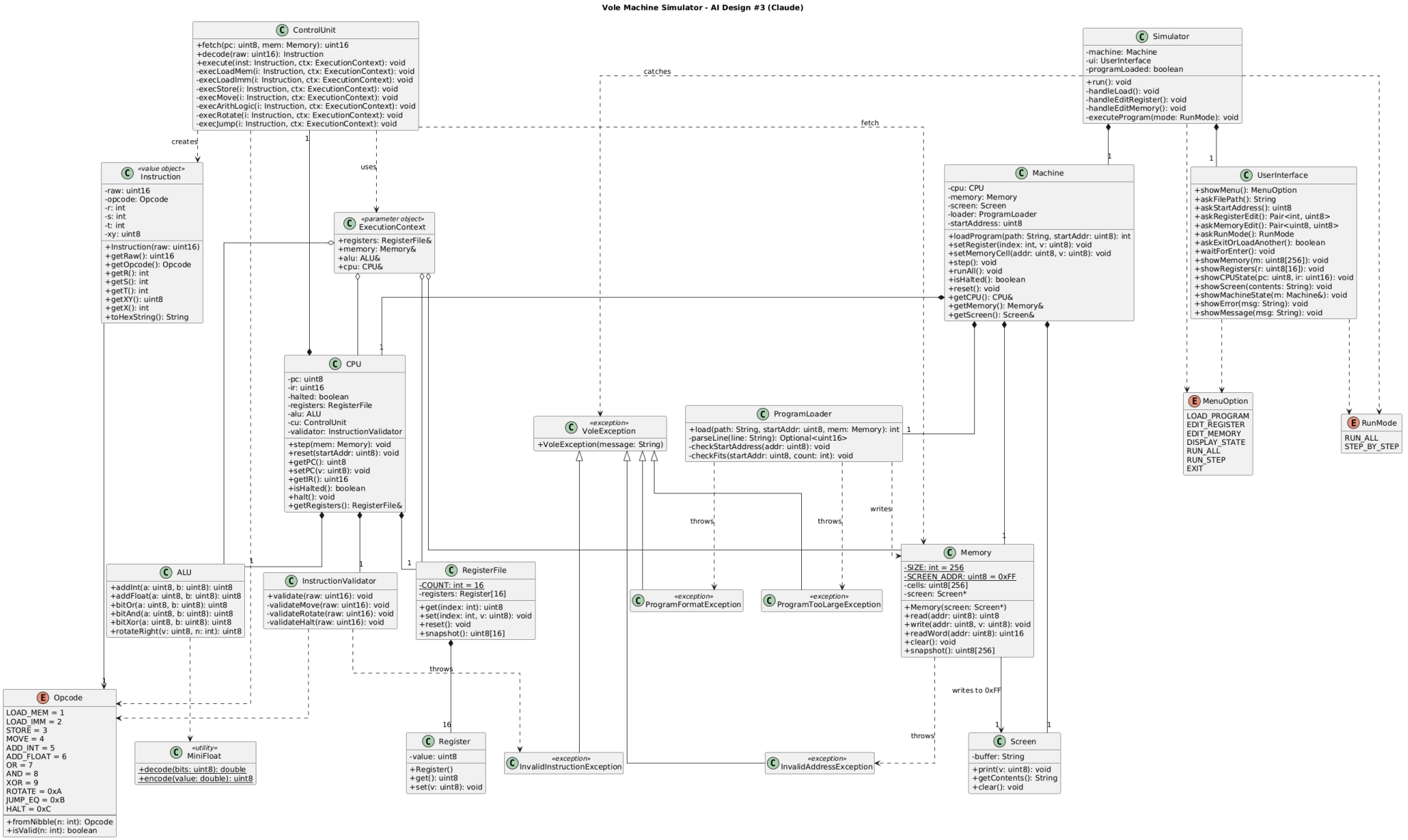


Figure 4. Anthropic Claude: robust, type-safe architecture (AI design #3)

PlantUML source code (ai3_claude_design.puml, 265 lines)

```
@startuml VoleSimulator
title Vole Machine Simulator - AI Design #3 (Claude)

skinparam classAttributeIconSize 0
skinparam linetype ortho
hide empty members

' ==================== ENUMS ====================
enum Opcode {
  LOAD_MEM = 1
  LOAD_IMM = 2
  STORE = 3
  MOVE = 4
  ADD_INT = 5
  ADD_FLOAT = 6
  OR = 7
  AND = 8
  XOR = 9
  ROTATE = 0xA
  JUMP_EQ = 0xB
  HALT = 0xC
  --
  +fromNibble(n: int): Opcode
  +isValid(n: int): boolean
}

enum MenuOption {
  LOAD_PROGRAM
  EDIT_REGISTER
  EDIT_MEMORY
  DISPLAY_STATE
  RUN_ALL
  RUN_STEP
  EXIT
}

enum RunMode {
  RUN_ALL
  STEP_BY_STEP
}

' ==================== EXCEPTIONS ====================
class VoleException <<exception>> {
  +VoleException(message: String)
}
class InvalidInstructionException <<exception>>
class InvalidAddressException <<exception>>
class ProgramFormatException <<exception>>
class ProgramTooLargeException <<exception>>

VoleException <|-- InvalidInstructionException
VoleException <|-- InvalidAddressException
VoleException <|-- ProgramFormatException
VoleException <|-- ProgramTooLargeException

' ==================== HARDWARE: STORAGE ====================
class Register {
  -value: uint8
  +Register()
  +get(): uint8
  +set(v: uint8): void
}

class RegisterFile {
  -{static} COUNT: int = 16
  -registers: Register[16]
  +get(index: int): uint8
  +set(index: int, v: uint8): void
  +reset(): void
  +snapshot(): uint8[16]
}

class Memory {
  -{static} SIZE: int = 256
  -{static} SCREEN_ADDR: uint8 = 0xFF
  -cells: uint8[256]
  -screen: Screen*
  +Memory(screen: Screen*)
  +read(addr: uint8): uint8
  +write(addr: uint8, v: uint8): void
  +readWord(addr: uint8): uint16
  +clear(): void
  +snapshot(): uint8[256]
}

class Screen {
  -buffer: String
  +print(v: uint8): void
  +getContents(): String
  +clear(): void
}

' ==================== HARDWARE: COMPUTATION ====================
class ALU {
  +addInt(a: uint8, b: uint8): uint8
  +addFloat(a: uint8, b: uint8): uint8
  +bitOr(a: uint8, b: uint8): uint8
  +bitAnd(a: uint8, b: uint8): uint8
  +bitXor(a: uint8, b: uint8): uint8
  +rotateRight(v: uint8, n: int): uint8
}

class MiniFloat <<utility>> {
  +{static} decode(bits: uint8): double
  +{static} encode(value: double): uint8
}

class Instruction <<value object>> {
  -raw: uint16
  -opcode: Opcode
  -r: int
  -s: int
  -t: int
  -xy: uint8
  +Instruction(raw: uint16)
  +getRaw(): uint16
  +getOpcode(): Opcode
  +getR(): int
  +getS(): int
  +getT(): int
  +getXY(): uint8
  +getX(): int
  +toHexString(): String
}

class InstructionValidator {
  +validate(raw: uint16): void
  -validateMove(raw: uint16): void
  -validateRotate(raw: uint16): void
  -validateHalt(raw: uint16): void
}

class ControlUnit {
  +fetch(pc: uint8, mem: Memory): uint16
  +decode(raw: uint16): Instruction
  +execute(inst: Instruction, ctx: ExecutionContext): void
  -execLoadMem(i: Instruction, ctx: ExecutionContext): void
  -execLoadImm(i: Instruction, ctx: ExecutionContext): void
  -execStore(i: Instruction, ctx: ExecutionContext): void
  -execMove(i: Instruction, ctx: ExecutionContext): void
  -execArithLogic(i: Instruction, ctx: ExecutionContext): void
  -execRotate(i: Instruction, ctx: ExecutionContext): void
  -execJump(i: Instruction, ctx: ExecutionContext): void
}

class ExecutionContext <<parameter object>> {
  +registers: RegisterFile&
  +memory: Memory&
  +alu: ALU&
  +cpu: CPU&
}

class CPU {
  -pc: uint8
  -ir: uint16
  -halted: boolean
  -registers: RegisterFile
  -alu: ALU
  -cu: ControlUnit
  -validator: InstructionValidator
  +step(mem: Memory): void
  +reset(startAddr: uint8): void
  +getPC(): uint8
  +setPC(v: uint8): void
  +getIR(): uint16
  +isHalted(): boolean
  +halt(): void
  +getRegisters(): RegisterFile&
}

class Machine {
  -cpu: CPU
  -memory: Memory
  -screen: Screen
  -loader: ProgramLoader
  -startAddress: uint8
  +loadProgram(path: String, startAddr: uint8): int
  +setRegister(index: int, v: uint8): void
  +setMemoryCell(addr: uint8, v: uint8): void
  +step(): void
  +runAll(): void
  +isHalted(): boolean
  +reset(): void
  +getCPU(): CPU&
  +getMemory(): Memory&
  +getScreen(): Screen&
}

class ProgramLoader {
  +load(path: String, startAddr: uint8, mem: Memory): int
  -parseLine(line: String): Optional<uint16>
  -checkStartAddress(addr: uint8): void
  -checkFits(startAddr: uint8, count: int): void
}

' ==================== APPLICATION LAYER ====================
class UserInterface {
  +showMenu(): MenuOption
  +askFilePath(): String
  +askStartAddress(): uint8
  +askRegisterEdit(): Pair<int, uint8>
```

```
  +askMemoryEdit(): Pair<uint8, uint8>
  +askRunMode(): RunMode
  +askExitOrLoadAnother(): boolean
  +waitForEnter(): void
  +showMemory(m: uint8[256]): void
  +showRegisters(r: uint8[16]): void
  +showCPUState(pc: uint8, ir: uint16): void
  +showScreen(contents: String): void
  +showMachineState(m: Machine&): void
  +showError(msg: String): void
  +showMessage(msg: String): void
}

class Simulator {
  -machine: Machine
  -ui: UserInterface
  -programLoaded: boolean
  +run(): void
  -handleLoad(): void
  -handleEditRegister(): void
  -handleEditMemory(): void
  -executeProgram(mode: RunMode): void
}

' ==================== RELATIONSHIPS ====================
' Composition
Machine *-- "1" CPU
Machine *-- "1" Memory
Machine *-- "1" Screen
CPU *-- "1" RegisterFile
CPU *-- "1" ALU
CPU *-- "1" ControlUnit
```

```
CPU *-- "1" InstructionValidator
RegisterFile *-- "16" Register
Simulator *-- "1" Machine
Simulator *-- "1" UserInterface
Machine *-- "1" ProgramLoader

' Association
Memory --> "1" Screen : writes to 0xFF
Instruction --> "1" Opcode

' Dependencies
ControlUnit ..> Instruction : creates
ControlUnit ..> Opcode
ControlUnit ..> ExecutionContext : uses
ControlUnit ..> Memory : fetch
ExecutionContext o-- RegisterFile
ExecutionContext o-- Memory
ExecutionContext o-- ALU
ExecutionContext o-- CPU
ALU ..> MiniFloat
InstructionValidator ..> Opcode
InstructionValidator ..> InvalidInstructionException : throws
ProgramLoader ..> Memory : writes
ProgramLoader ..> ProgramFormatException : throws
ProgramLoader ..> ProgramTooLargeException : throws
Memory ..> InvalidAddressException : throws
Simulator ..> MenuOption
Simulator ..> RunMode
UserInterface ..> MenuOption
UserInterface ..> RunMode
Simulator ..> VoleException : catches
@enduml
```

RE-RUN OF THE SAME PROMPT (21 OCTOBER 2026)

The original chat transcript of the session above was not kept. To document the interaction, the same prompt (Section 3.1) was sent again to Claude in a new session. This re-run is a separate response and its design differs from the original design in Figure 4; Sections 3.6 and 4 refer to the original design.

Your 11-class suggestion mirrors the hardware well, so I kept its overall shape. I made a few changes where the design would otherwise cause problems during the C++ implementation.

Changes to your initial suggestion

1. **No separate `Register` class.** A register is just one `uint8_t`. `RegisterFile` holds `uint8_t[16]` and exposes `get`/`set`, which avoids 16 tiny objects and keeps indexing simple.
2. **Added `Screen`.** Address FF is memory-mapped I/O, not an ordinary cell. `Memory::write(0xFF, v)` forwards the byte to `Screen`, so the control unit never needs to know about the mapping.
3. **Kept `CU` separate from `CPU`.** `CPU` owns the state (PC, IR, halted flag, registers) and runs the cycle in `step()`. `ControlUnit` only executes an already decoded `Instruction`, with one small private method per instruction family.
4. **`Instruction` is an immutable value object.** `Instruction::decode(raw)` splits the four nibbles and throws `InvalidInstructionException` for an unknown opcode or an illegal operand pattern (for example `40RS` with a non-zero second nibble, `AR0X` with a non-zero third nibble, or `C` with non-zero operands). So the invalid-program requirement lives in one place.
5. **Small exception hierarchy.** `VoleException` with `InvalidInstructionException`, `InvalidAddressException` and `ProgramFormatException`. `Simulator` catches `VoleException` and reports it through `UserInterface::showError()`, instead of passing error codes through every layer.
6. **Enums instead of raw numbers:** `Opcode`, `MenuChoice` and `RunMode`.
7. **C++ types:** `uint8_t` for registers, memory cells, PC and addresses; `uint16_t` for IR and raw instructions.

Execution cycle (one `CPU::step`)

1. `ir = memory.readWord(pc)`
2. `inst = Instruction::decode(ir)` (validation happens here)
3. `pc = pc + 2`
4. `controlUnit.execute(inst, *this, memory)`

The PC is advanced **before** execute on purpose: a taken `B` (JUMP) instruction then simply overwrites the PC with XY, so no special case is needed.

Points to watch

- Because FF is the screen, a program must fit in addresses 00 to FE and an instruction cannot start at FE (it would span FF). `ProgramLoader` should reject such programs with `InvalidAddressException`, and also reject odd start addresses.
- `Memory` receives the `Screen` it forwards to, so `Machine` must create `Screen` before `Memory`.
- Opcode 6 needs an 8-bit floating-point format (1 sign bit, 3 exponent bits in excess-4, 4 mantissa bits). I kept the decode/encode helpers private inside `ALU`; if they grow, they can move to a small utility class.

The PlantUML code of the response is shown in the "PlantUML source code" part of this section.

PlantUML source code (Claude's PlantUML code, re-run, 210 lines)

```
@startuml
title Vole Machine Simulator - Class Diagram (Claude, re-run)

skinparam classAttributeIconSize 0
skinparam packageStyle rectangle

package "Exceptions" {
    class VoleException <<exception>> {
        + VoleException(message: string)
        + what(): const char*
    }
    class InvalidInstructionException <<exception>>
    class InvalidAddressException <<exception>>
    class ProgramFormatException <<exception>>

    VoleException <|-- InvalidInstructionException
    VoleException <|-- InvalidAddressException
    VoleException <|-- ProgramFormatException
}

package "Hardware" {
    class RegisterFile {
        - regs: uint8_t[16]
        + get(index: uint8_t): uint8_t
        + set(index: uint8_t, value: uint8_t): void
        + reset(): void
        + snapshot(): array<uint8_t, 16>
    }

    class Screen {
        - buffer: string
        + write(value: uint8_t): void
        + getOutput(): string
        + clear(): void
    }

    class Memory {
        + {static} SCREEN_ADDRESS: uint8_t = 0xFF
        - cells: uint8_t[256]
        - screen: Screen*
        + Memory(screen: Screen*)
        + read(address: uint8_t): uint8_t
        + write(address: uint8_t, value: uint8_t): void
        + readWord(address: uint8_t): uint16_t
        + clear(): void
        + snapshot(): array<uint8_t, 256>
    }

    class ALU {
        + addInt(a: uint8_t, b: uint8_t): uint8_t
        + addFloat(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseOr(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseAnd(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseXor(a: uint8_t, b: uint8_t): uint8_t
        + rotateRight(value: uint8_t, bits: uint8_t): uint8_t
        - decodeFloat(bits: uint8_t): double
        - encodeFloat(value: double): uint8_t
    }
}

package "InstructionSet" {
    enum Opcode {
        LOAD_MEM
        LOAD_IMM
        STORE
        MOVE
        ADD_INT
        ADD_FLOAT
        OR
        AND
        XOR
        ROTATE
        JUMP_EQ
        HALT
    }

    class Instruction <<value object>> {
        - raw: uint16_t
        - opcode: Opcode
        - r: uint8_t
        - s: uint8_t
        - t: uint8_t
        - xy: uint8_t
        + {static} decode(raw: uint16_t): Instruction
        + getRaw(): uint16_t
        + getOpcode(): Opcode
        + getR(): uint8_t
        + getS(): uint8_t
        + getT(): uint8_t
        + getXY(): uint8_t
        + getX(): uint8_t
        + toString(): string
    }
}

package "Processor" {
    class ControlUnit {
        + execute(inst: Instruction, cpu: CPU&, mem: Memory&):
            void
        - executeLoad(inst: Instruction, cpu: CPU&, mem:
            Memory&): void
        - executeStore(inst: Instruction, cpu: CPU&, mem:
            Memory&): void
        - executeMove(inst: Instruction, cpu: CPU&): void
        - executeArithmeticLogic(inst: Instruction, cpu: CPU&):
            void
        - executeRotate(inst: Instruction, cpu: CPU&): void
        - executeJump(inst: Instruction, cpu: CPU&): void
        - executeHalt(cpu: CPU&): void
    }

    class CPU {
        - pc: uint8_t
        - ir: uint16_t
        - halted: bool
        - registers: RegisterFile
        - alu: ALU
        - controlUnit: ControlUnit
        + reset(startAddress: uint8_t): void
        + step(mem: Memory&): bool
        + run(mem: Memory&): void
        + getPC(): uint8_t
        + setPC(address: uint8_t): void
        + getIR(): uint16_t
        + isHalted(): bool
        + setHalted(value: bool): void
        + getRegisters(): RegisterFile&
        + getALU(): ALU&
    }
}

package "Loader" {
    class ProgramLoader {
        + loadFromFile(path: string, mem: Memory&, startAddress:
            uint8_t): size_t
        + isValidStartAddress(address: uint8_t): bool
        - parseLine(line: string, out: uint16_t&): bool
    }
}

package "Application" {
    enum MenuChoice {
        LOAD_PROGRAM
        EDIT_REGISTER
        EDIT_MEMORY
        RUN
        EXIT
    }

    enum RunMode {
        RUN_ALL
        STEP_BY_STEP
    }

    class Machine {
        - screen: Screen
        - memory: Memory
        - cpu: CPU
        + loadProgram(path: string, startAddress: uint8_t): void
        + setRegister(index: uint8_t, value: uint8_t): void
        + setMemoryCell(address: uint8_t, value: uint8_t): void
        + step(): bool
        + runAll(): void
        + reset(): void
        + getCPU(): CPU&
        + getMemory(): Memory&
        + getScreen(): Screen&
    }

    class UserInterface {
        + showMainMenu(): MenuChoice
        + askRunMode(): RunMode
        + askFilePath(): string
        + askStartAddress(): uint8_t
        + askRegisterIndex(): uint8_t
        + askMemoryAddress(): uint8_t
        + askByteValue(): uint8_t
        + showMachineState(cpu: CPU&, mem: Memory&, screen:
            Screen&): void
        + showError(message: string): void
        + waitForEnter(): void
        + askExitOrLoadAnother(): bool
    }

    class Simulator {
        - machine: Machine
        - ui: UserInterface
        + run(): void
        - handleLoad(): void
        - handleEditRegister(): void
        - handleEditMemory(): void
        - handleExecute(mode: RunMode): void
    }
}

' Relationships
Simulator *-- Machine
Simulator *-- UserInterface
Machine *-- Screen
Machine *-- Memory
Machine *-- CPU
Machine ..> ProgramLoader : loads with
```

```
Memory ..> Screen : forwards writes to FF
CPU *-- RegisterFile
CPU *-- ALU
CPU *-- ControlUnit
CPU ..> Instruction : decodes
ControlUnit ..> Instruction : executes
ControlUnit ..> Memory : reads and writes
```

```
Instruction ..> Opcode
Instruction ..> InvalidInstructionException : throws
ProgramLoader ..> Memory : writes into
ProgramLoader ..> ProgramFormatException : throws
UserInterface ..> MenuChoice
UserInterface ..> RunMode
@enduml
```

3.5 AI #4: GitHub Copilot

ORIGINAL RESPONSE

Here's a complete **PlantUML class diagram design** for the Vole Machine Simulator, followed by an explanation of the design choices. (The PlantUML code of the response is shown in the "PlantUML source code" part of this section.)

Design Choices Explained

1. **Register & RegisterFile**
   - Each register is a simple 8-bit container.
   - RegisterFile manages all 16 registers and provides indexed access.
   - Composition is used because registers cannot exist outside the RegisterFile.
2. **Memory**
   - A 256-byte array with read/write methods.
   - Includes a `displayMatrix()` method to show memory in a 16x16 grid.
3. **ALU**
   - Encapsulates arithmetic and logic operations.
   - Keeps CPU logic clean by separating computation from control.
4. **Instruction**
   - Represents a decoded instruction (opcode + operands).
   - Parsing logic ensures raw 16-bit values are converted into structured instructions.
5. **Control Unit (CU)**
   - Handles the fetch-decode-execute cycle.
   - Depends on ALU, RegisterFile, and Memory to execute instructions.
6. **CPU**
   - Central component with PC, IR, halt flag.
   - Owns RegisterFile, ALU, and CU.
   - Provides `step()` (single instruction) and `run()` (full program execution).
7. **ProgramLoader**
   - Reads program files, parses instructions, and loads them into memory.
   - Ensures programs start at even addresses.
8. **Machine**
   - Represents the hardware abstraction.
   - Owns CPU and Memory.
   - Provides methods to reset, load programs, and run them.
9. **UserInterface**
   - Handles all console I/O.
   - Displays memory, registers, IR, PC, and screen output.
   - Prompts user for overwriting values.
10. **Simulator**
    - Top-level controller.
    - Coordinates Machine and UserInterface.
    - Implements the main loop (menu, execution, reload/exit).

Improvements Over Initial Suggestion

- Added **display methods** in RegisterFile and Memory for visualization.
- Made **Instruction** a dedicated class to encapsulate parsing and validation.
- ControlUnit explicitly depends on ALU, RegisterFile, and Memory for clarity.
- Machine provides **reset()** for reusability when loading multiple programs.
- UserInterface includes **screen output display** for memory-mapped I/O.

Would you like me to also **illustrate the execution flow** (sequence diagram) for one instruction cycle (fetch-decode-execute), so you can visualize how the classes interact dynamically?

UML CLASS DIAGRAM

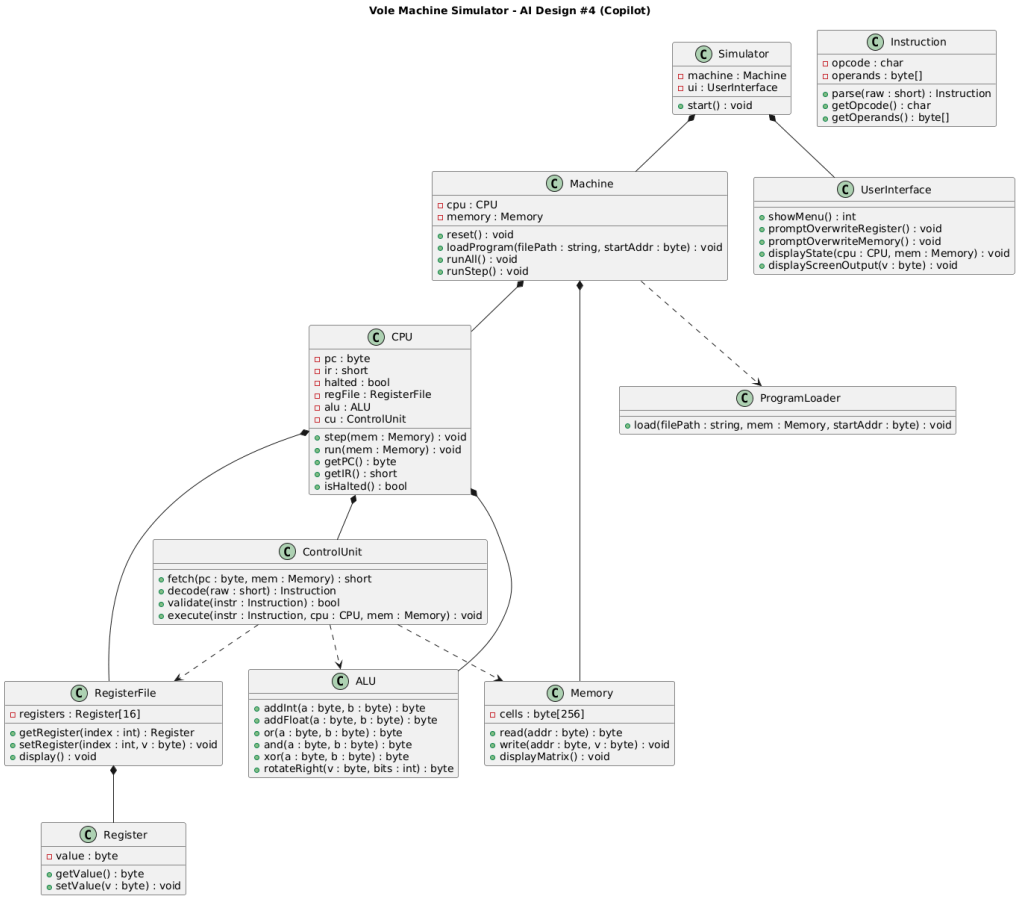


Figure 5. GitHub Copilot: minimalist 11-class architecture (AI design #4)

PlantUML source code (ai4_copilot_design.puml, 103 lines)

```
@startuml
title Vole Machine Simulator - AI Design #4 (Copilot)

class Register {
  - value : byte
  + getValue() : byte
  + setValue(v : byte) : void
}

class RegisterFile {
  - registers : Register[16]
  + getRegister(index : int) : Register
  + setRegister(index : int, v : byte) : void
  + display() : void
}

class Memory {
  - cells : byte[256]
  + read(addr : byte) : byte
  + write(addr : byte, v : byte) : void
  + displayMatrix() : void
}

class ALU {
  + addInt(a : byte, b : byte) : byte
  + addFloat(a : byte, b : byte) : byte
  + or(a : byte, b : byte) : byte
  + and(a : byte, b : byte) : byte
  + xor(a : byte, b : byte) : byte
  + rotateRight(v : byte, bits : int) : byte
}

class Instruction {
  - opcode : char
  - operands : byte[]
  + parse(raw : short) : Instruction
  + getOpcode() : char
  + getOperands() : byte[]
}

class ControlUnit {
  + fetch(pc : byte, mem : Memory) : short
  + decode(raw : short) : Instruction
  + validate(instr : Instruction) : bool
  + execute(instr : Instruction, cpu : CPU, mem : Memory) : void
}

class CPU {
  - pc : byte
  - ir : short
  - halted : bool
  - regFile : RegisterFile
  - alu : ALU
  - cu : ControlUnit
  + step(mem : Memory) : void
  + run(mem : Memory) : void
  + getPC() : byte
  + getIR() : short
  + isHalted() : bool
}

class ProgramLoader {
  + load(filePath : string, mem : Memory, startAddr : byte) :
      void
}

class Machine {
  - cpu : CPU
  - memory : Memory
  + reset() : void
  + loadProgram(filePath : string, startAddr : byte) : void
  + runAll() : void
  + runStep() : void
}

class UserInterface {
  + showMenu() : int
  + promptOverwriteRegister() : void
  + promptOverwriteMemory() : void
  + displayState(cpu : CPU, mem : Memory) : void
  + displayScreenOutput(v : byte) : void
}

class Simulator {
  - machine : Machine
  - ui : UserInterface
  + start() : void
}

' Relationships
Machine *-- CPU
Machine *-- Memory
CPU *-- RegisterFile
CPU *-- ALU
CPU *-- ControlUnit
RegisterFile *-- Register
Machine ..> ProgramLoader
Simulator *-- Machine
Simulator *-- UserInterface
ControlUnit ..> ALU
ControlUnit ..> RegisterFile
ControlUnit ..> Memory

@enduml
```

3.6 Comparison of the Four Designs

Dimension	Gemini (#1)	ChatGPT (#2)	Claude (#3)	Copilot (#4)
Philosophy	Lean, packages	Rich, SRP	Production-ready	Simple, conservative
Total elements	9	15 (13 classes + 2 enums)	24	11
Separate CU / RegisterFile	RegisterFile yes, CU merged in CPU	Yes	Yes	Yes
Screen class	Yes	Yes	Yes	No
Program class	No	Yes	No	No
Opcode enum	No	Yes	Yes, with methods	No
Exception hierarchy	No	No	Yes (5 classes)	No
MiniFloat utility	No	No	Yes	No
InstructionValidator	No	No	Yes	No
ExecutionContext	No	No	Yes	No
UML packages	5	None	Layers (described)	None
PC advanced before execute	Not mentioned	Not mentioned	Yes (explicit)	Not mentioned
Main weakness	CPU does everything; no Opcode enum	No exceptions or validator	May be over-engineered	No enums, no Screen

Ranking: 1st Claude, 2nd ChatGPT, 3rd Gemini, 4th Copilot. The designs differ mainly in how much structure they add: Gemini and Copilot stay close to the 11-class suggestion, ChatGPT adds Screen and Program, and Claude adds a full robustness layer (exceptions, validation, MiniFloat, ExecutionContext).

4 Final Enhanced Design

4.1 Why Claude's Design Was Selected

1. **Most comprehensive architecture.** 24 elements (14 classes, 5 exceptions, 1 utility, 1 value object, 3 enums) cover every part of the simulator without gaps.
2. **Exception hierarchy.** The only design with VoleException and four specialized exceptions (invalid instruction, invalid address, program format, program too large), so errors are classified and reported clearly.
3. **MiniFloat utility.** Opcode 6 needs 8-bit floating-point addition (1 sign bit, 3 exponent bits in excess-4, 4 mantissa bits). Isolating it keeps the ALU clean and the format testable.
4. **ExecutionContext.** A parameter object bundling registers, memory, ALU and CPU, so the control unit stays stateless and has short method signatures.
5. **InstructionValidator.** Validation is separated from decoding (single responsibility): Instruction parses, the validator decides whether it is legal.
6. **PC incremented before execute.** Claude noted that PC must advance before execution; otherwise a successful JUMP would be overwritten by PC += 2. No other AI mentioned this subtle bug.
7. **Immutable Instruction.** Once parsed, an Instruction cannot change, which prevents accidental mutation during execution.

4.2 What Was Enhanced from the Other AIs

Enhancement	Source	What was added
Package organization	Gemini (#1)	6 packages: Exceptions, Hardware, InstructionSet, Processor, Loader, Application
C++ types	Gemini (#1)	uint8_t, uint16_t, bool and string instead of generic byte and short
Notes on all classes	All	PlantUML notes explaining each class's role and behavior
display() / displayMatrix()	ChatGPT (#2), Copilot (#4)	Display methods for console output

4.3 Refinement Dialog with Gemini 3.8 Flash

The four designs and the enhancement goal were given to a different AI, Gemini 3.8 Flash, in a refinement session. The prompt and the response are reproduced below, followed by the design it proposed (Figure 6).

Prompt given to the AI (Gemini 3.8 Flash)

```
We have evaluated 4 AI designs:
- Gemini (AI #1): Introduced strong 5-package architecture and eliminated primitive wrappers.
- ChatGPT (AI #2): Strong argument for separating CU execution engine from CPU register state,
  and introduced explicit Screen buffering.
- Claude (AI #3): Industrial-grade design featuring Opcode enums, MiniFloat IEEE/Brookshear
  isolation, ExecutionContext parameter object, and VoleException hierarchy.
- Copilot (AI #4): Minimalist 11-class model with efficient atomic memory operations
  (readWord, writeBlock).

Select the best design as a baseline, explain why, and synthesize the ultimate enhanced
architecture incorporating the strengths of all models.
```

Response

Baseline Selection: Claude (AI #3) was unequivocally chosen as the primary foundation because it represents the highest engineering quality:

1. **Defensive Robustness:** Dedicated `VoleException` hierarchy (`InvalidInstructionException`, `InvalidAddressException`, `ProgramFormatException`).
2. **Decoupled Execution Flow:** An `ExecutionContext` struct bundle that eliminates parameter bloating when dispatching instructions.
3. **Mathematical Correctness:** Isolating Brookshear 8-bit floating-point bit unpacker/packer inside `MiniFloat`.

What Was Enhanced to Reach the Final Production Blueprint:

1. **Adopted Gemini's Modular Packaging:** Embedded Claude's 24 components into 6 rigorous UML packages: `Hardware`, `InstructionSet`, `Processor`, `Loader`, `Application`, and `Exceptions`.
2. **Adopted ChatGPT's Explicit Screen Buffer & CU Decoupling:** Replaced primitive stdout writes with a formal `Screen` class with scrolling line history, keeping `CU` as the execution engine and `CPU` as the state holder.
3. **Adopted Copilot's Memory Utilities:** Added `readWord(addr)` and `writeBlock(addr, data)` to `Memory` for atomic 16-bit instruction fetches without manually orchestrating two byte reads.
4. **C++ Native Types:** Upgraded all signatures to `uint8_t`, `uint16_t`, `size_t`, and `const std::string&`.

Here is the **Definitive Final Enhanced PlantUML Design** (its PlantUML code is shown in the "PlantUML source code" part of this section).

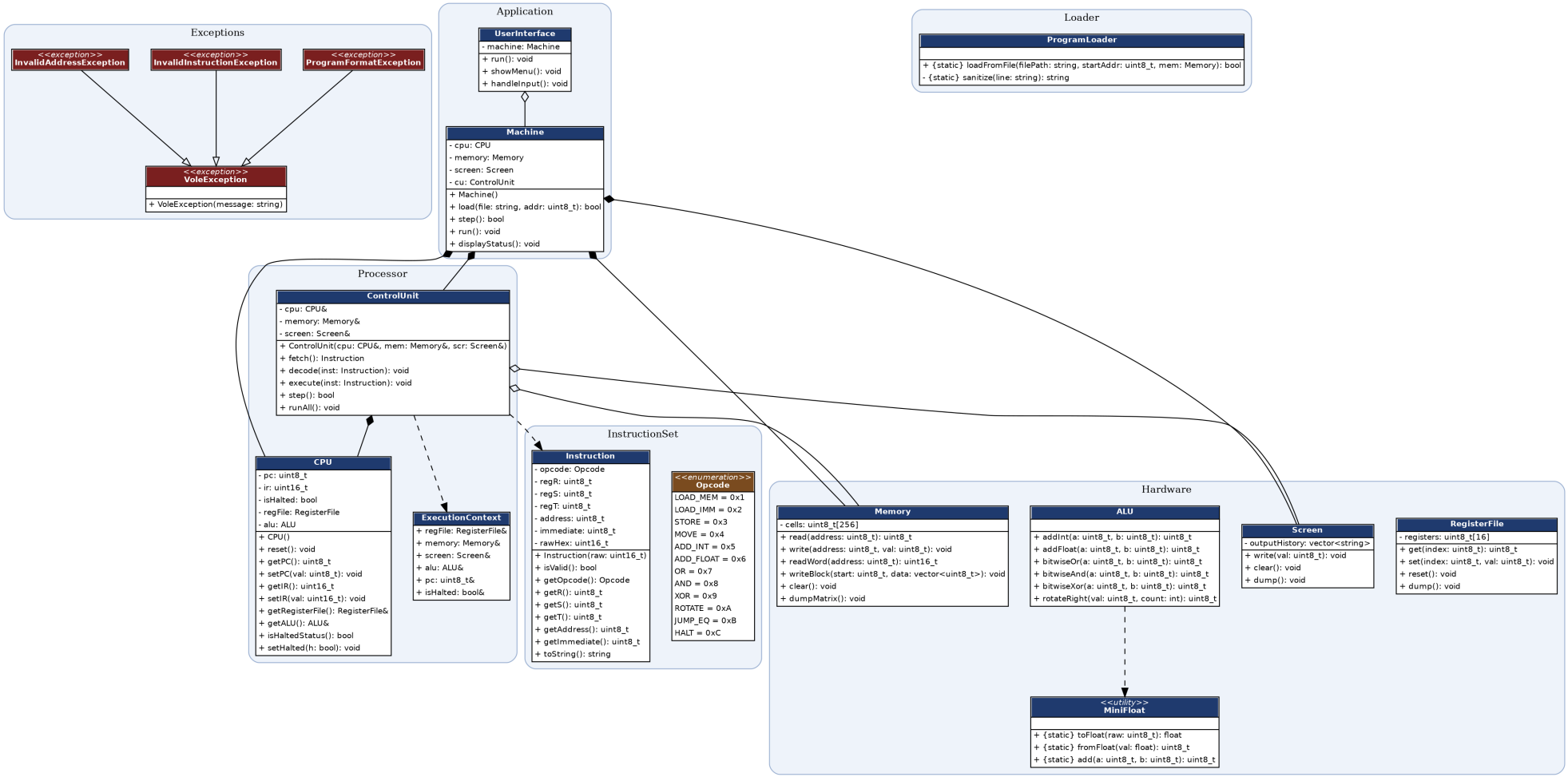


Figure 6. Final design proposed by Gemini 3.8 Flash, rendered with Graphviz from the PlantUML code

PlantUML source code (Gemini 3.8 Flash's final PlantUML code, 181 lines)

```
@startuml VoleSimulator_FinalDesign
title Vole Machine Simulator - Final Enhanced Design

skinparam classAttributeIconSize 0
skinparam classFontSize 11
skinparam linetype ortho
skinparam packageStyle rectangle

package "Exceptions" {
    class VoleException <<exception>> {
        + VoleException(message: string)
    }
    class InvalidInstructionException <<exception>>
    class InvalidAddressException <<exception>>
    class ProgramFormatException <<exception>>

    VoleException <|-- InvalidInstructionException
    VoleException <|-- InvalidAddressException
    VoleException <|-- ProgramFormatException
}

package "Hardware" {
    class RegisterFile {
        - registers: uint8_t[16]
        + get(index: uint8_t): uint8_t
        + set(index: uint8_t, val: uint8_t): void
        + reset(): void
        + dump(): void
    }

    class Memory {
        - cells: uint8_t[256]
        + read(address: uint8_t): uint8_t
        + write(address: uint8_t, val: uint8_t): void
        + readWord(address: uint8_t): uint16_t
        + writeBlock(start: uint8_t, data: vector<uint8_t>):
            void
        + clear(): void
        + dumpMatrix(): void
    }

    class MiniFloat <<utility>> {
        + {static} toFloat(raw: uint8_t): float
        + {static} fromFloat(val: float): uint8_t
        + {static} add(a: uint8_t, b: uint8_t): uint8_t
    }

    class ALU {
        + addInt(a: uint8_t, b: uint8_t): uint8_t
        + addFloat(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseOr(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseAnd(a: uint8_t, b: uint8_t): uint8_t
        + bitwiseXor(a: uint8_t, b: uint8_t): uint8_t
        + rotateRight(val: uint8_t, count: int): uint8_t
    }

    class Screen {
        - outputHistory: vector<string>
        + write(val: uint8_t): void
        + clear(): void
        + dump(): void
    }
}

package "InstructionSet" {
    enum Opcode {
        LOAD_MEM = 0x1
        LOAD_IMM = 0x2
        STORE = 0x3
        MOVE = 0x4
        ADD_INT = 0x5
        ADD_FLOAT = 0x6
        OR = 0x7
        AND = 0x8
        XOR = 0x9
        ROTATE = 0xA
        JUMP_EQ = 0xB
        HALT = 0xC
    }

    class Instruction {
        - opcode: Opcode
        - regR: uint8_t
        - regS: uint8_t
        - regT: uint8_t
        - address: uint8_t
        - immediate: uint8_t
        - rawHex: uint16_t
        + Instruction(raw: uint16_t)
        + isValid(): bool
        + getOpcode(): Opcode
        + getR(): uint8_t
        + getS(): uint8_t
        + getT(): uint8_t
        + getAddress(): uint8_t
        + getImmediate(): uint8_t
        + toString(): string
    }
}

package "Processor" {
    class ExecutionContext {
        + regFile: RegisterFile&
        + memory: Memory&
        + screen: Screen&
        + alu: ALU&
        + pc: uint8_t&
        + isHalted: bool&
    }

    class CPU {
        - pc: uint8_t
        - ir: uint16_t
        - isHalted: bool
        - regFile: RegisterFile
        - alu: ALU
        + CPU()
        + reset(): void
        + getPC(): uint8_t
        + setPC(val: uint8_t): void
        + getIR(): uint16_t
        + setIR(val: uint16_t): void
        + getRegisterFile(): RegisterFile&
        + getALU(): ALU&
        + isHaltedStatus(): bool
        + setHalted(h: bool): void
    }

    class ControlUnit {
        - cpu: CPU&
        - memory: Memory&
        - screen: Screen&
        + ControlUnit(cpu: CPU&, mem: Memory&, scr: Screen&)
        + fetch(): Instruction
        + decode(inst: Instruction): void
        + execute(inst: Instruction): void
        + step(): bool
        + runAll(): void
    }
}

package "Loader" {
    class ProgramLoader {
        + {static} loadFromFile(filePath: string, startAddr:
            uint8_t, mem: Memory): bool
        - {static} sanitize(line: string): string
    }
}

package "Application" {
    class Machine {
        - cpu: CPU
        - memory: Memory
        - screen: Screen
        - cu: ControlUnit
        + Machine()
        + load(file: string, addr: uint8_t): bool
        + step(): bool
        + run(): void
        + displayStatus(): void
    }

    class UserInterface {
        - machine: Machine
        + run(): void
        + showMenu(): void
        + handleInput(): void
    }
}

' Relations
ALU ..> MiniFloat
ControlUnit *-- CPU
ControlUnit o-- Memory
ControlUnit o-- Screen
ControlUnit ..> ExecutionContext
ControlUnit ..> Instruction
Machine *-- CPU
Machine *-- Memory
Machine *-- Screen
Machine *-- ControlUnit
UserInterface o-- Machine
@enduml
```

4.4 Final Design Adopted by the Team

The final design adopted by the team (Figure 7) follows the AI's proposal and also keeps `InstructionValidator`, `MenuOption`, `RunMode` and `ProgramTooLargeException` from Claude's design, giving 27 elements in total.

STRUCTURE

```
Exceptions (5)     VoleException (base), InvalidInstructionException, InvalidAddressException,
                   ProgramFormatException, ProgramTooLargeException
Hardware (4)       Register, RegisterFile, Memory (readWord, snapshot), Screen (memory-mapped I/O at FF)
InstructionSet     Instruction (immutable, self-parsing), InstructionValidator, Opcode (enum, fromNibble)
Processor (4+1)    ALU, MiniFloat (utility), ExecutionContext, ControlUnit (stateless), CPU (PC, IR, halted)
Loader (1)         ProgramLoader (file reading, parsing, memory loading)
Application (3+2)  Machine (facade), UserInterface, Simulator, MenuOption (enum), RunMode (enum)

Total: 19 classes/structs + 3 enums + 5 exceptions = 27 elements
```

EXECUTION FLOW

```
Simulator.run()
  UI.showMenu() -> MenuOption
  handleLoad():  UI.askFilePath(), UI.askStartAddress()
                 Machine.loadProgram():  ProgramLoader.load() -> Memory.write();  CPU.reset(startAddr)
                 UI.showMachineState()
  executeProgram(RUN_ALL | STEP_BY_STEP):  Machine.runAll() / runStep()
    loop: CPU.step(memory)
      1. CU.fetch(pc, mem) -> IR            4. PC += 2          (before execute, so JUMP can overwrite it)
      2. Validator.validate(IR)             5. CU.execute(instr, ctx)  -> ALU, RegisterFile, Memory,
      3. CU.decode(IR) -> Instruction                                     Screen.print() on STORE to FF
  UI.showMachineState()   (memory matrix, PC, IR, 16 registers, screen)
```

UML CLASS DIAGRAM

The final diagram is on the next page (Figure 7) and its PlantUML source follows it.

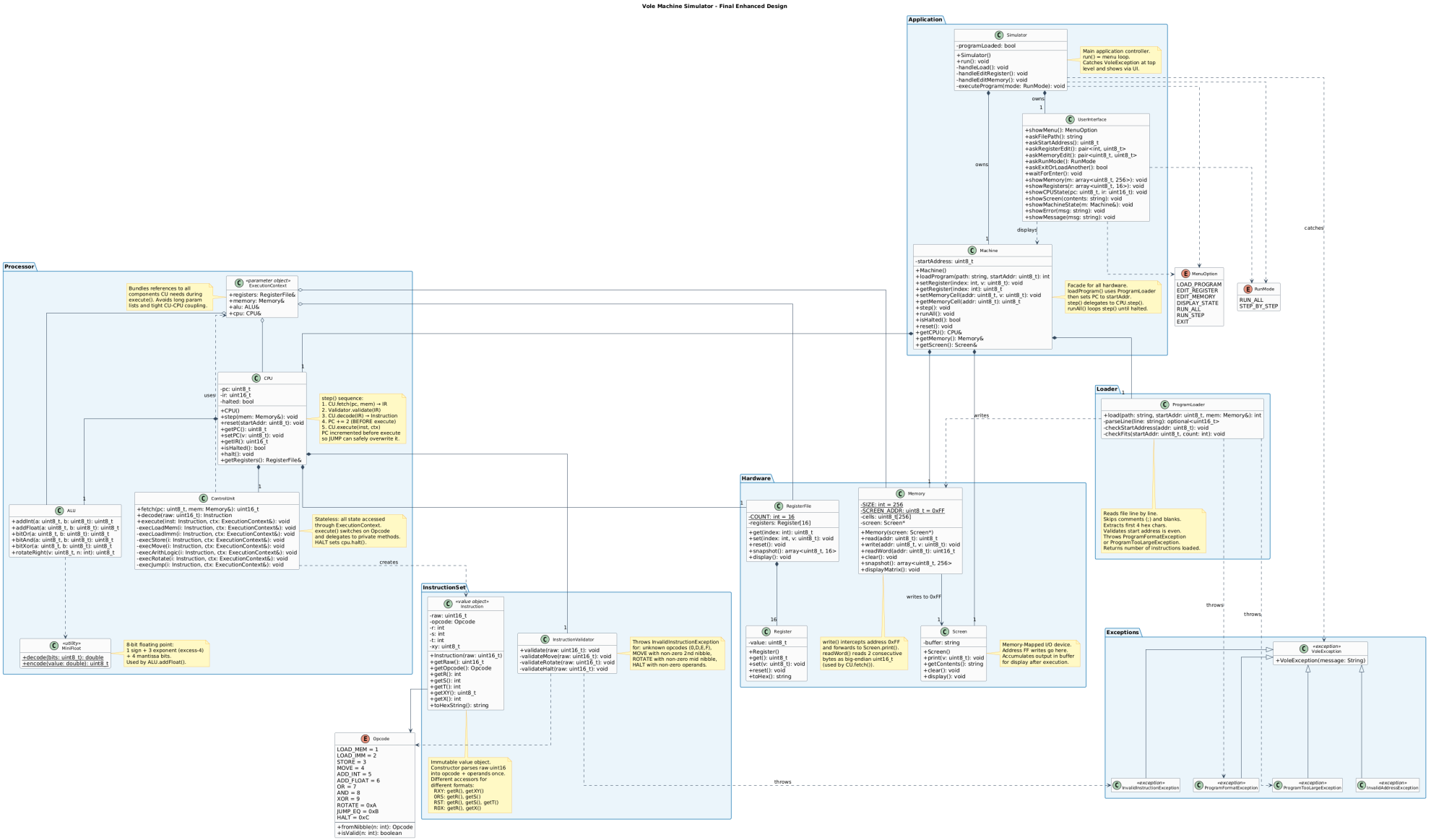


Figure 7. Final design adopted by the team: six-package modular architecture

PlantUML source code (final_design.puml, 375 lines)

```
@startuml VoleSimulator_FinalDesign
title Vole Machine Simulator - Final Enhanced Design

skinparam classAttributeIconSize 0
skinparam classFontSize 11
skinparam linetype ortho
skinparam classBackgroundColor #FAFAFA
skinparam classBorderColor #2C3E50
skinparam arrowColor #34495E
skinparam noteBackgroundColor #FFF9C4
skinparam noteBorderColor #F9A825
skinparam packageBackgroundColor #EBF5FB
skinparam packageBorderColor #2980B9
hide empty members

' ENUMS

enum Opcode {
  LOAD_MEM = 1
  LOAD_IMM = 2
  STORE = 3
  MOVE = 4
  ADD_INT = 5
  ADD_FLOAT = 6
  OR = 7
  AND = 8
  XOR = 9
  ROTATE = 0xA
  JUMP_EQ = 0xB
  HALT = 0xC
  --
  +fromNibble(n: int): Opcode
  +isValid(n: int): boolean
}

enum MenuOption {
  LOAD_PROGRAM
  EDIT_REGISTER
  EDIT_MEMORY
  DISPLAY_STATE
  RUN_ALL
  RUN_STEP
  EXIT
}

enum RunMode {
  RUN_ALL
  STEP_BY_STEP
}

' EXCEPTION HIERARCHY
package "Exceptions" {
  class VoleException <<exception>> {
    +VoleException(message: String)
  }
  class InvalidInstructionException <<exception>>
  class InvalidAddressException <<exception>>
  class ProgramFormatException <<exception>>
  class ProgramTooLargeException <<exception>>

  VoleException <|-- InvalidInstructionException
  VoleException <|-- InvalidAddressException
  VoleException <|-- ProgramFormatException
  VoleException <|-- ProgramTooLargeException
}

' PACKAGE: Hardware — Storage
package "Hardware" {
  class Register {
    -value: uint8_t
    +Register()
    +get(): uint8_t
    +set(v: uint8_t): void
    +reset(): void
    +toHex(): string
  }

  class RegisterFile {
    -{static} COUNT: int = 16
    -registers: Register[16]
    +get(index: int): uint8_t
    +set(index: int, v: uint8_t): void
    +reset(): void
    +snapshot(): array<uint8_t, 16>
    +display(): void
  }

  RegisterFile *-- "16" Register

  class Memory {
    -{static} SIZE: int = 256
    -{static} SCREEN_ADDR: uint8_t = 0xFF
    -cells: uint8_t[256]
    -screen: Screen*
    +Memory(screen: Screen*)
    +read(addr: uint8_t): uint8_t
    +write(addr: uint8_t, v: uint8_t): void
    +readWord(addr: uint8_t): uint16_t
    +clear(): void
    +snapshot(): array<uint8_t, 256>
    +displayMatrix(): void
  }
  note bottom of Memory
    write() intercepts address 0xFF
    and forwards to Screen.print().
    readWord() reads 2 consecutive
    bytes as big-endian uint16_t
    (used by CU.fetch()).
  end note

  class Screen {
    -buffer: string
    +Screen()
    +print(v: uint8_t): void
    +getContents(): string
    +clear(): void
    +display(): void
  }
  note right of Screen
    Memory-Mapped I/O device.
    Address FF writes go here.
    Accumulates output in buffer
    for display after execution.
  end note

  Memory --> "1" Screen : "writes to 0xFF"
}

' PACKAGE: InstructionSet
package "InstructionSet" {
  class Instruction <<value object>> {
    -raw: uint16_t
    -opcode: Opcode
    -r: int
    -s: int
    -t: int
    -xy: uint8_t
    +Instruction(raw: uint16_t)
    +getRaw(): uint16_t
    +getOpcode(): Opcode
    +getR(): int
    +getS(): int
    +getT(): int
    +getXY(): uint8_t
    +getX(): int
    +toHexString(): string
  }
  note bottom of Instruction
    Immutable value object.
    Constructor parses raw uint16
    into opcode + operands once.
    Different accessors for
    different formats:
      RXY: getR(), getXY()
      0RS: getR(), getS()
      RST: getR(), getS(), getT()
      R0X: getR(), getX()
  end note

  Instruction --> Opcode

  class InstructionValidator {
    +validate(raw: uint16_t): void
    -validateMove(raw: uint16_t): void
    -validateRotate(raw: uint16_t): void
    -validateHalt(raw: uint16_t): void
  }
  note right of InstructionValidator
    Throws InvalidInstructionException
    for: unknown opcodes (0,D,E,F),
    MOVE with non-zero 2nd nibble,
    ROTATE with non-zero mid nibble,
    HALT with non-zero operands.
  end note

  InstructionValidator ..> Opcode
  InstructionValidator ..> InvalidInstructionException : throws
}

' PACKAGE: Processor
package "Processor" {
  class ALU {
    +addInt(a: uint8_t, b: uint8_t): uint8_t
    +addFloat(a: uint8_t, b: uint8_t): uint8_t
    +bitOr(a: uint8_t, b: uint8_t): uint8_t
    +bitAnd(a: uint8_t, b: uint8_t): uint8_t
    +bitXor(a: uint8_t, b: uint8_t): uint8_t
    +rotateRight(v: uint8_t, n: int): uint8_t
  }

  class MiniFloat <<utility>> {
    +{static} decode(bits: uint8_t): double
    +{static} encode(value: double): uint8_t
  }
  note right of MiniFloat
    8-bit floating point:
    1 sign + 3 exponent (excess-4)
    + 4 mantissa bits.
    Used by ALU.addFloat().
  end note

  ALU ..> MiniFloat
```

```
  class ExecutionContext <<parameter object>> {
    +registers: RegisterFile&
    +memory: Memory&
    +alu: ALU&
    +cpu: CPU&
  }
  note left of ExecutionContext
    Bundles references to all
    components CU needs during
    execute(). Avoids long param
    lists and tight CU-CPU coupling.
  end note

  ExecutionContext o-- RegisterFile
  ExecutionContext o-- Memory
  ExecutionContext o-- ALU
  ExecutionContext o-- CPU

  class ControlUnit {
    +fetch(pc: uint8_t, mem: Memory&): uint16_t
    +decode(raw: uint16_t): Instruction
    +execute(inst: Instruction, ctx: ExecutionContext&): void
    -execLoadMem(i: Instruction, ctx: ExecutionContext&): void
    -execLoadImm(i: Instruction, ctx: ExecutionContext&): void
    -execStore(i: Instruction, ctx: ExecutionContext&): void
    -execMove(i: Instruction, ctx: ExecutionContext&): void
    -execArithLogic(i: Instruction, ctx: ExecutionContext&):
        void
    -execRotate(i: Instruction, ctx: ExecutionContext&): void
    -execJump(i: Instruction, ctx: ExecutionContext&): void
  }
  note right of ControlUnit
    Stateless: all state accessed
    through ExecutionContext.
    execute() switches on Opcode
    and delegates to private methods.
    HALT sets cpu.halt().
  end note

  ControlUnit ..> Instruction : creates
  ControlUnit ..> ExecutionContext : uses

  class CPU {
    -pc: uint8_t
    -ir: uint16_t
    -halted: bool
    +CPU()
    +step(mem: Memory&): void
    +reset(startAddr: uint8_t): void
    +getPC(): uint8_t
    +setPC(v: uint8_t): void
    +getIR(): uint16_t
    +isHalted(): bool
    +halt(): void
    +getRegisters(): RegisterFile&
  }
  note right of CPU
    step() sequence:
    1. CU.fetch(pc, mem) → IR
    2. Validator.validate(IR)
    3. CU.decode(IR) → Instruction
    4. PC += 2 (BEFORE execute)
    5. CU.execute(inst, ctx)
    PC incremented before execute
    so JUMP can safely overwrite it.
  end note

  CPU *-- "1" RegisterFile
  CPU *-- "1" ALU
  CPU *-- "1" ControlUnit
  CPU *-- "1" InstructionValidator
}

' PACKAGE: Loader
package "Loader" {
  class ProgramLoader {
    +load(path: string, startAddr: uint8_t, mem: Memory&): int
    -parseLine(line: string): optional<uint16_t>
    -checkStartAddress(addr: uint8_t): void
    -checkFits(startAddr: uint8_t, count: int): void
  }
  note bottom of ProgramLoader
    Reads file line by line.
    Skips comments (;) and blanks.
    Extracts first 4 hex chars.
    Validates start address is even.
    Throws ProgramFormatException
    or ProgramTooLargeException.
    Returns number of instructions loaded.
  end note

  ProgramLoader ..> Memory : writes
  ProgramLoader ..> ProgramFormatException : throws
  ProgramLoader ..> ProgramTooLargeException : throws
}

' PACKAGE: Application
package "Application" {
  class Machine {
    -startAddress: uint8_t
    +Machine()
    +loadProgram(path: string, startAddr: uint8_t): int
    +setRegister(index: int, v: uint8_t): void
    +getRegister(index: int): uint8_t
    +setMemoryCell(addr: uint8_t, v: uint8_t): void
    +getMemoryCell(addr: uint8_t): uint8_t
    +step(): void
    +runAll(): void
    +isHalted(): bool
    +reset(): void
    +getCPU(): CPU&
    +getMemory(): Memory&
    +getScreen(): Screen&
  }
  note right of Machine
    Facade for all hardware.
    loadProgram() uses ProgramLoader
    then sets PC to startAddr.
    step() delegates to CPU.step().
    runAll() loops step() until halted.
  end note

  Machine *-- "1" CPU
  Machine *-- "1" Memory
  Machine *-- "1" Screen
  Machine *-- "1" ProgramLoader

  class UserInterface {
    +showMenu(): MenuOption
    +askFilePath(): string
    +askStartAddress(): uint8_t
    +askRegisterEdit(): pair<int, uint8_t>
    +askMemoryEdit(): pair<uint8_t, uint8_t>
    +askRunMode(): RunMode
    +askExitOrLoadAnother(): bool
    +waitForEnter(): void
    +showMemory(m: array<uint8_t, 256>): void
    +showRegisters(r: array<uint8_t, 16>): void
    +showCPUState(pc: uint8_t, ir: uint16_t): void
    +showScreen(contents: string): void
    +showMachineState(m: Machine&): void
    +showError(msg: string): void
    +showMessage(msg: string): void
  }

  UserInterface ..> Machine : displays
  UserInterface ..> MenuOption
  UserInterface ..> RunMode

  class Simulator {
    -programLoaded: bool
    +Simulator()
    +run(): void
    -handleLoad(): void
    -handleEditRegister(): void
    -handleEditMemory(): void
    -executeProgram(mode: RunMode): void
  }
  note right of Simulator
    Main application controller.
    run() = menu loop.
    Catches VoleException at top
    level and shows via UI.
  end note

  Simulator *-- "1" Machine : owns
  Simulator *-- "1" UserInterface : owns
  Simulator ..> VoleException : catches
  Simulator ..> MenuOption
  Simulator ..> RunMode
}

@enduml
```

5 Sample Vole Machine Programs

Two programs were written in the required file format and are used to evaluate the machine. Both write their result to the screen with a store to address FF. The comments show each instruction's address in square brackets; both programs were traced step by step.

5.1 Program 1: Sum of 1 to 5

File `programs/program1.vole`. Adds 1 + 2 + 3 + 4 + 5 in a loop and writes **15 (0x0F)** to the screen. The loop ends when the counter reaches 6, and the unconditional back-jump uses `B00A` (R0 always equals R0).

```
; ------------------------------------------------------------
; Vole Machine Program 1: Sum of 1 + 2 + 3 + 4 + 5
; Result: 15 (0x0F) is written to the screen (address FF)
; ------------------------------------------------------------
2000 ; [00] R0 = 0x00 (zero, used by the jump comparisons)
2100 ; [02] R1 = 0x00 (running sum)
2201 ; [04] R2 = 0x01 (counter i)
2301 ; [06] R3 = 0x01 (step)
2406 ; [08] R4 = 0x06 (stop value)
5112 ; [0A] loop: R1 = R1 + R2
5223 ; [0C] R2 = R2 + R3 (i++)
9524 ; [0E] R5 = R2 XOR R4 (zero when i == 6)
B514 ; [10] if R5 == R0, jump to 14 (done)
B00A ; [12] R0 == R0 always: jump back to 0A
31FF ; [14] store R1 to the screen (FF)
C000 ; [16] halt
```

5.2 Program 2: Maximum of Two Signed Numbers

File `programs/program2.vole`. Computes a - b with two's complement (NOT b + 1, then ADD), tests the sign bit with AND 0x80, and writes the larger of a = 0x18 and b = 0x24, which is **0x24**, to the screen. Changing the two immediate values in the first lines tests other inputs.

```
; ------------------------------------------------------------
; Vole Machine Program 2: Maximum of two signed numbers
; a = 0x18 (24), b = 0x24 (36). The larger value is shown on screen
; ------------------------------------------------------------
2000 ; [00] R0 = 0x00
2118 ; [02] R1 = 0x18 (a)
2224 ; [04] R2 = 0x24 (b)
24FF ; [06] R4 = 0xFF
9324 ; [08] R3 = R2 XOR R4 (NOT b)
2401 ; [0A] R4 = 0x01
5334 ; [0C] R3 = R3 + R4 (-b, two's complement)
5513 ; [0E] R5 = R1 + R3 (a - b)
2680 ; [10] R6 = 0x80 (sign mask)
8756 ; [12] R7 = R5 AND R6 (nonzero when a < b)
B718 ; [14] if R7 == R0 (a >= b), jump to 18
4021 ; [16] otherwise R1 = R2 (b is the maximum)
31FF ; [18] store R1 (the maximum) to the screen (FF)
C000 ; [1A] halt
```

6 Video Link & Lessons Learned

VIDEO PRESENTATION LINK

Watch the 10-minute presentation video (Google Drive)

`https://drive.google.com/file/d/1tbjRVmPvINsFUEC15UT9zDK8YTUTTR7y/view?usp=drive_link`

KEY LESSONS

- **Value of using several AIs.** Different models have different design biases: Gemini favored structure and packages, ChatGPT added peripherals and data classes, Claude added defensive robustness, and Copilot stayed close to our design. Comparing them exposed dimensions that one prompt would have missed.
- **Hardware decoupling.** Keeping the control unit (logic) separate from the CPU (state) simplifies future extensions such as pipelining.
- **Subtle details matter.** The PC must advance before execute so JUMP works, and the screen is mapped to address FF, not 00. Catching these at design time avoids bugs in Parts 2 and 3.
- **Concrete C++ types.** Designing with uint8_t and uint16_t, validation and error classes early reduces friction when implementing the simulator.